SAPIENZA
UNIVERSITÀ DI ROMA

Concurrent Systems

UNIVERSITY OF ROME "LA SAPIENZA"

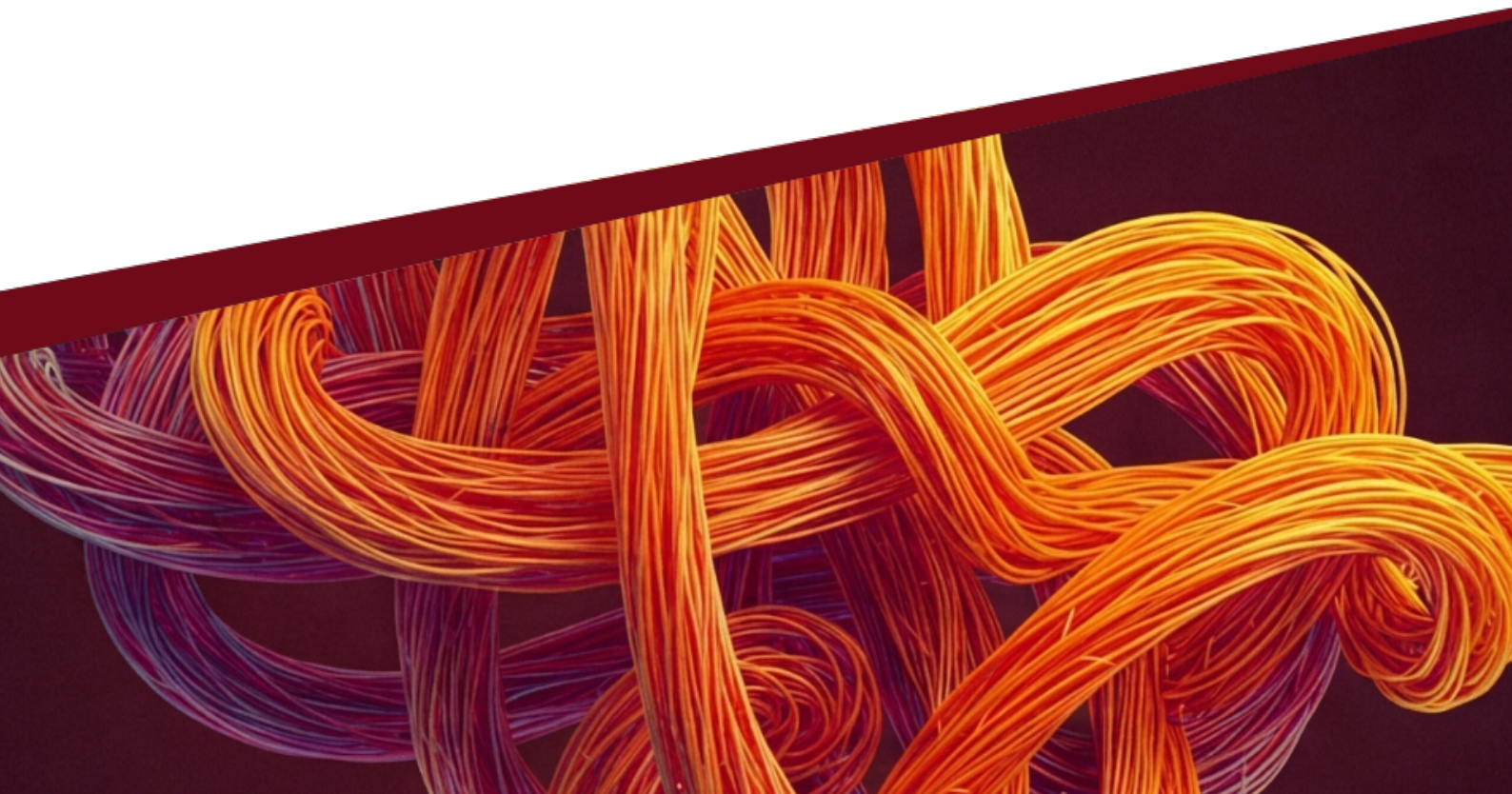
FACULTY OF INFORMATION ENGINEERING, COMPUTER SCIENCE AND STATISTICS

MASTER'S DEGREE IN COMPUTER SCIENCE

Author
LORENZO SABATINO

Based on the lectures of
PROF. DANIELE GORLA

April 7, 2026



Preface

Hello everyone!

My name is Lorenzo Sabatino and I am a student of the Computer Science course at La Sapienza University of Rome. During my academic career, I have faced numerous exams by transcribing the notes taken in class and transforming them into real study books.

I wanted to share this material with you because I believe it can be a valid help for all Computer Science students, both those who are beginners and those who are already advanced in their studies. I have written and formatted these texts with care, using the LaTeXmarkup language to ensure clarity and readability. I have also enriched the content with diagrams and schemes and I have integrated ideas taken from online lessons and textbooks.

However, it is important to underline that these books do not replace the classroom lessons and are not free of errors. My intention was to provide additional support for studying, but it is essential to participate in the lessons and delve into the concepts with the guidance of the teachers. Also, please note that due to the ever-evolving nature of computer science, some information may become outdated over time.

Nevertheless, I decided to share these books because I believe they can be of help to many students and I hope you will find them useful in your academic journey.

If you wish to contact me with any questions, comments or corrections, you can write to me at scrittinformatica@gmail.com. I will be happy to answer your requests.

Thank you for choosing to read these books and good luck with your studies!

Contents

Chapter 1

Concurrency

To understand concurrency, we must first distinguish between the abstract idea of an algorithm and its physical realization. A *sequential algorithm* serves as the "idea"—it is a formal description of the behavior of an abstract sequential state machine, detailing the transitions that must be executed in order. When this algorithm is translated into a specific programming language, it becomes a program, or the "text". The *process* represents the "action"; it is the dynamic entity generated when a program is executed on a concrete machine. At any given moment, a process is defined by its state, which includes its program counter and register values. A sequential process, often referred to as a thread, is managed by a single control flow and program counter.

Concurrency arises when a set of these sequential state machines run simultaneously and interact through a communication medium, such as shared memory. This creates a multiprocess program or concurrent system. There are significant advantages to this approach: it allows for the combination of work from different processes to solve distinct tasks in parallel, and it simplifies complex programming tasks by breaking them down into more manageable, simpler components.

1.1 Features of a Concurrent System

Concurrent systems can be categorized by several key assumptions regarding their reliability and timing. In our study, we focus on systems that are reliable, asynchronous, and utilize shared memory.

- *Reliability:* Every process is assumed to be reliable, meaning it correctly executes the code of its corresponding algorithm.
- *Asynchrony:* An asynchronous system makes no timing assumptions. Each process operates on its own independent clock and proceeds at an arbitrary speed.
- *Shared Memory:* While every process has its own local memory accessible only to itself, the system includes shared registers or memory locations that can be accessed by every process in the program.

Regarding hardware, we often simplify the model by assuming there is one processor available for every process, allowing them to execute in parallel. However, in systems where there are fewer processors than processes, a hidden scheduler is responsible for assigning processors to processes.

1.2 Process Synchronization: Cooperation and Competition

Synchronization occurs whenever the progress of one process depends on the behavior of others. These interactions generally fall into two categories: cooperation and competition.

1.2.1 Cooperation

In *cooperation*, different processes work together to ensure they all succeed in their respective tasks.

- *Rendezvous (Barrier):* A synchronization barrier is a set of control points, one for each process involved. A process is only permitted to pass its control point once every other involved process has reached its own. This forces processes to stop and wait for one another, ensuring they are synchronized at a specific stage of execution.
- *Producer-Consumer:* This common pattern involves two types of processes: the producer, which generates data items, and the consumer, which utilizes them. This cooperation is governed by two strict constraints: only data that has been produced can be consumed, and every item produced must be consumed exactly once.

1.2.2 Competition

Competition occurs when multiple processes attempt to execute specific actions or access a shared resource, but only one (or a limited number) can succeed at a time. A classic example is two processes attempting to withdraw 1M€ from the same bank account simultaneously.

```
1 function withdraw() {
2     x := account.read();
3     if x ≥ 1M then account.write(x - 1M)
4 }
```

Listing 1.1: Withdraw from account balance.

Consider the `withdraw()` function that reads the account balance, checks if it is at least 1M€, and then writes the new balance. While individual read and write operations are typically atomic to happen instantaneously at a single point in time—their sequential composition within the function is not.

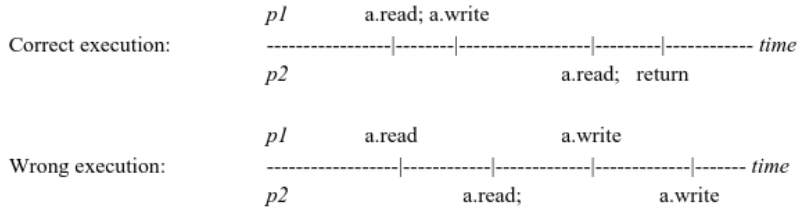


Figure 1.1: Process executions of `withdraw()` function.

In a correct execution, process p_1 might complete its read and write before p_2 begins. However, in a wrong execution caused by interleaving, both p_1 and p_2 might read the account balance while it is still 1M€. Both will see the condition as true and proceed to write a new balance of 0€, effectively withdrawing 2M€ from an account that only contained 1M€.

Chapter 2

Mutual Exclusion (MUTEX)

To resolve the issues presented by competition, we use *Mutual Exclusion*, which ensures that specific parts of code are executed as if they were atomic, without intermission from other processes. This is necessary not only for competition but also for cooperation when accessing shared resources.

A *Critical Section* (C.S.) is a part of the code that must be executed by at most one process at a time. The Mutual Exclusion problem involves designing an entry protocol (often called `lock`) and an exit protocol (`unlock`) to encapsulate the critical section.

We operate under two main assumptions:

1. All critical sections will eventually terminate if executed by a single process.
2. The code is well-formed, meaning processes always follow the sequence of locking, executing the critical section, and then unlocking.

2.1 Safety and Liveness Properties

The effectiveness of any synchronization solution is judged by two types of properties: Safety and Liveness.

- *Safety:* "Nothing bad ever happens." In the context of MUTEX, the safety property is mutual exclusion itself: at most one process is in the critical section at any given time.
- *Liveness:* "Something good eventually happens." Liveness ensures the system remains meaningful; for instance, a system that simply forbids all activity would be safe but useless because no process would ever enter the critical section.

For Mutual Exclusion, there are three main tiers of liveness:

1. *Deadlock-freedom:* This is the weakest liveness property. It states that if one or more processes are trying to enter the critical section, eventually at least one of them will succeed. It focuses on system-wide progress rather than individual process progress.
2. *Starvation-freedom:* A stronger property ensuring that every process that attempts to enter the critical section will eventually be granted access. This is more meaningful from the perspective of an individual "client" process.
3. *Bounded bypass:* The strongest property, stating that there is a limit—defined by a function $f(n)$—to how many times a process can be "bypassed" by others before it is granted entry to the critical section.

These properties form a strict hierarchy where

$$\text{Bounded bypass} \implies \text{Starvation-freedom} \implies \text{Deadlock-freedom}.$$

The inclusions are strict, meaning the reverse is not necessarily true. For example, Deadlock-freedom does not imply Starvation-freedom. Consider three processes (p_1,p_2,p_3) repeatedly trying to enter a critical section. It is possible for p_2 and p_3 to alternate winning the competition, continually

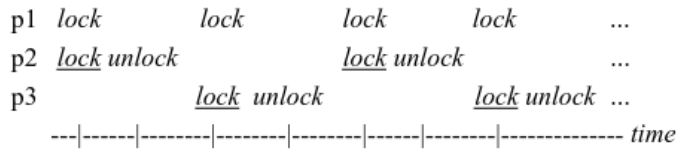


Figure 2.1: Hierarchy of liveness properties process executions (underlined actions succeed).

bypassing p_1. In this scenario, the system is deadlock-free because progress is being made (p_2 and p_3 are entering the C.S.), but it is not starvation-free because p_1 is locked out forever.

Similarly, Starvation-freedom does not imply Bounded bypass. While starvation-freedom guarantees that a process will eventually enter (the number of bypasses is finite), it does not provide a specific mathematical bound ($f(n)$) on how many times that process might be overtaken before its turn arrives.

2.2 Atomic Read/Write Registers

In the study of concurrent systems, the complexity and correctness of an algorithm often depend on the level of atomicity provided by the underlying hardware. We begin by considering Atomic Read/Write (R/W) registers. These are basic storage units that support two fundamental operations: `READ` and `WRITE`. For a register to be considered "atomic," it must satisfy specific properties that allow us to reason about it as if operations occur instantaneously.

An atomic register ensures that every operation invocation behaves as though it takes place at a single, distinct point on a timeline. Formally, there exists a function t that maps each operation invocation to a positive real number. This point t must fall within the actual time interval of the operation—between its start and its end. Furthermore, this function is injective, meaning that two operations cannot happen at the same logical moment. From a data consistency perspective, an atomic register guarantees that any `READ` operation returns the value written by the most recent `WRITE` that preceded it, or the initial value if no `WRITE` has yet occurred.

These registers are further classified by their accessibility. A Single-Read/Single-Write (SRSW) register is the most restrictive, allowing only one specific process to read and one specific process to write. As we scale, we encounter Single-Read/Multiple-Write (SRMW), Multiple-Read/Single-Write (MRSW), and finally, Multiple-Read/Multiple-Write (MRMW) registers, which allow any process to perform the designated operations.

2.3 The Peterson Algorithm

The *Peterson algorithm* is a classic solution to the mutual exclusion (MUTEX) problem. To understand its final form, we can examine the failures of simpler attempts to coordinate two processes, p_0 and p_1.

In a first attempt, we might use a single shared variable, `LAST`, to act as an indicator. When a process i wants to enter the critical section, it sets `LAST` to its own ID and waits until `LAST` is no longer equal to i.

```
1 lock(i) :=
2     LAST ← i
3     wait LAST ≠ i
4     return
```

```
1 unlock(i) :=
2     return
```

Listing 2.1: Wrong Peterson algorithm.

While this ensures that two processes cannot both enter the critical section (since the variable can only hold one value), it suffers from liveness flaw: if only one process ever tries to lock, it will set the variable to its own ID and wait forever for a second process that never arrives to change it.

In a second attempt, we might use a `FLAG` array where `FLAG[i]` indicates process i's intent to enter the critical section. A process sets its flag to "up" and waits for the other process's flag to be "down."

```
1 Initialize FLAG[0] and FLAG[1] to down
```

```
1 lock(i) :=
2     FLAG[i] ← up
3     wait FLAG[1-i] = down
4     return
```

```
1 unlock(i) :=
2     FLAG[i] ← down
3     return
```

Listing 2.2: Wrong Peterson algorithm.

This approach also fails due to deadlock. If both processes simultaneously raise their flags, they will both enter the wait state, each waiting for the other to lower their flag, which will never happen.

The correct Peterson algorithm combines these two ideas: the `FLAG` array to indicate intent and the `LAST` variable to break ties.

```
1 Initialize FLAG[0] and FLAG[1] to down
```

```
1 lock(i) :=
2     FLAG[i] ← up
3     LAST ← i
4     wait (FLAG[1-i] = down
5         OR LAST ≠ i)
6     return
```

```
1 unlock(i) :=
2     FLAG[i] ← down
3     return
```

Listing 2.3: Correct solution to the Peterson algorithm.

In this protocol, a process p_i first signals its intent by raising its flag. Crucially, it then sets `LAST` to its own index, essentially saying "I am interested, but the other process can go first if it wants to." It only enters the critical section if either the other process is uninterested (its flag is down) or the other process have setting `LAST` to its own index.

Demonstration 2.3.1 – MUTEX proof for Peterson algorithm

To prove that Peterson's algorithm satisfies MUTEX, we can use a proof by contradiction. Assume that both p_0 and p_1 are in the critical section simultaneously. For p_0 to have entered, its wait condition must have been false. This could happen in two scenarios:

1. `FLAG[1]` was down: This implies that when p_0 checked the flag, p_1 had not yet raised its flag or had already exited. However, for p_1 to be in the critical section, it must have eventually raised its flag and set `LAST` to 1. If p_1 raised its flag after p_0 checked it, p_1 would then set `LAST` to 1 and find p_0's flag already up, causing p_1 to wait.

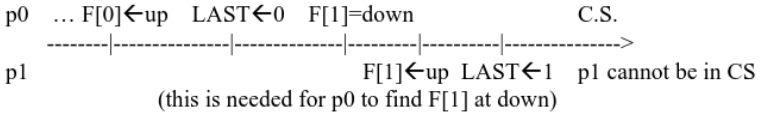


Figure 2.2: First case of the MUTEX proof for Peterson Algorithm.

2. `LAST` was 1: For p_0 to see `LAST` as 1 (not 0), p_1 must have been the last one to write to that variable. If p_1 was the last to write, then p_1 would find `LAST == 1` and `FLAG[0] == up`, forcing p_1 to wait.

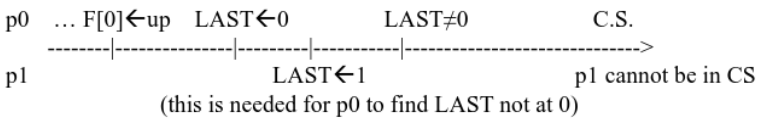


Figure 2.3: Second case of the MUTEX proof for Peterson Algorithm.

In both cases, it is impossible for both processes to pass the wait condition at the same time.

Demonstration 2.3.2 – Bounded Bypass proof for Peterson algorithm

Peterson's algorithm for two processes provides a bounded bypass of 1. This means that if p_0 is waiting to enter, p_1 can enter and exit the critical section at most once before p_0 is guaranteed entry.
If p_0 is blocked, it means `FLAG[1]` is "up" and `LAST` is 0. This implies p_1 is either in the critical section or about to enter. When p_1 exits, it sets `FLAG[1]` to "down." If p_1 tries to re-enter immediately, it must set `LAST` to 1. At that point, p_0 will see that `LAST` is no longer 0 and will be able to enter the critical section.

2.3.1 Peterson's Algorithm for n Processes

The algorithm can be generalized to n processes by visualizing it as a series of $n-1$ levels. A process must pass through each level—like a filter—to reach the critical section. At each level, at least one process is "left behind," effectively reducing the number of processes that can proceed to the next level.

The `FLAG` array now tracks which level a process has reached, and an array of `LAST` variables (one for each level) acts as the tie-breaker for that specific level.

```
1  Initialize FLAG[i] to 0, for all i
```

```
1  lock(i) :=
2      for lev = 1 to n-1 do
3          FLAG[i] ← lev
4          LAST[lev] ← i
5          wait (∀k≠i FLAG[k] < lev
6              OR LAST[lev] ≠ i)
7      return
```

```
1  unlock(i) :=
2      FLAG[i] ← 0
3      return
```

Listing 2.4: Peterson algorithm for n processes.

Lemma 2.3.1 – MUTEX proof for Peterson algorithm for n processes

The safety of the n-process algorithm rests on the lemma that for every level $\ell \in \{0, \ldots, n-1\}$, at most $n-\ell$ processes can be at that level. By the time we reach level $n-1$, only $(n-(n-1)) = 1$ process can remain, satisfying MUTEX.

Demonstration. This is proven by induction on ℓ. The base case $(\ell = 0)$ is trivial, as all n processes start at level 0. For the inductive step, we consider the last process p_x to write to `LAST`[ℓ+1]. For p_x to pass to level $\ell+1$, all other processes p_k must have a flag value lower than $\ell+1$. If p_x is blocked, and since at most $n-\ell$ processes reached level ℓ, there are at most $n-\ell-1$ processes that can potentially be at level $\ell+1$ while p_x waits.

Lemma 2.3.2 – Starvation freedom proof for Peterson algorithm for n processes

The algorithm is starvation-free, meaning every process at level ℓ eventually reaches level $\ell+1$ and, ultimately, the critical section.

Demonstration. This is proven by reverse induction from level $n-1$ down to 0. If a process p_x is blocked at level ℓ, it must be that `LAST`[ℓ+1] == x and some other process p_y has a flag value $\geq \ell+1$. By the inductive hypothesis, p_y will eventually progress and exit. If no other process enters level $\ell+1$ to change the `LAST` variable, p_x will eventually be the only process at that level and will pass the wait condition.

The n-process Peterson algorithm is robust but comes with specific memory and performance costs. It requires n MRSW registers for the `FLAG` array and $n-1$ MRMW registers for the `LAST` array. Each entry into the critical section requires $(n-1) \times (n+2)$ register accesses for locking and 1 access for unlocking.

While it guarantees starvation freedom, the n-process version does not satisfy bounded bypass.

Because a process can "sleep" arbitrarily long during its first wait, other processes can theoretically enter and exit the critical section an unbounded number of times before the sleeping process wakes up and progresses to the next level.

Finally, the algorithm is easily generalized to k-MUTEX, where up to k processes are allowed in the critical section simultaneously. This is achieved by simply reducing the number of levels in the loop from $n-1$ to $n-k$.

2.4 Concurrent Algorithms With Better Performance

Peterson's algorithm, while effective for two processes, becomes computationally expensive when generalized to n processes. In its standard n-process form, it requires a process to compete across $n-1$ levels, leading to a contention-free time complexity of $O(n^2)$. To optimize this, we can employ a tournament-based approach.

This approach utilizes a complete binary tree where each node represents a 2-process mutual exclusion algorithm (such as Peterson's). For n processes, the tree has a height of $\lceil \log_2 n \rceil$. A process wishing to enter the Critical Section (CS) must "win" a competition at each level of the tree, starting from a leaf and moving toward the root. Since each competition at a node has a constant cost, the total cost to reach the CS is reduced to $O(\log n)$.

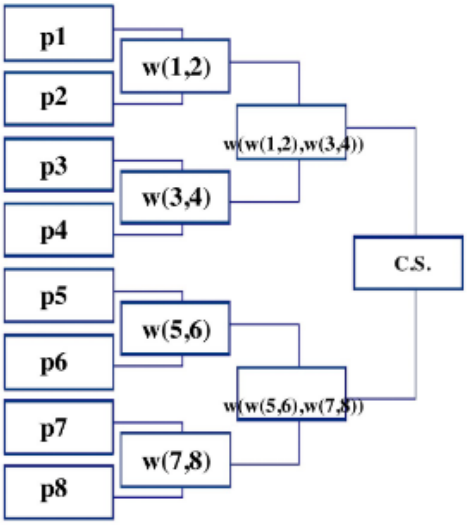


Figure 2.4: Tournament-based algorithm tree.

The quest for even greater efficiency leads to the exploration of $O(1)$ algorithms. Ideally, in a "contention-free" scenario (where only one process is active), the process should be able to enter its CS with a fixed, small number of register accesses.

The initial idea for such an algorithm involves two shared registers: `X` and `Y`. `X` is used to track the identity of the process currently attempting to lock, while `Y` acts as a "gatekeeper."

```
1  Initialize Y at ⊥, X at any value (e.g., 0)
```

```
1  lock(i) :=
2      X ← i
3      if Y ≠ ⊥ then FAIL
4          else Y ← i
5              if X = i then return
6                  else FAIL
```

```
1  unlock(i) :=
2      Y ← ⊥
3      return
```

Listing 2.5: Constant-time algorithm for n processes.

In a simplified version, a process i writes its ID to `X` and then checks `Y`. If `Y` is already set, the process `FAIL`. Otherwise, it sets `Y` to i and checks if `X` is still equal to i. If both conditions hold, the process enters the CS. Without contention, this requires only four register accesses.

However, this FAIL mechanism is problematic because it forces processes to repeatedly retry the lock, which can lead to a deadlock. Specifically, a scenario can occur where multiple processes interfere with each other's X and Y values such that no process ever successfully enters the CS.

2.4.1 Lamport's Fast MUTEX Algorithm

To resolve the deadlock issue while maintaining $O(1)$ complexity in the absence of contention, Leslie Lamport designed the Fast Mutex Algorithm. This algorithm uses an array of flags (FLAG[1..n]) and two shared registers, X and Y (with Y initialized to $\perp$).

```
Initialize Y at ⊥, X at any value (e.g., 0)
```

```
lock(i) :=
*   FLAG[i] ← up
    X ← i
    if Y ≠ ⊥ then FLAG[i] ← down
                  wait Y = ⊥
                  goto *
         else Y ← i
              if X = i then return
                   else FLAG[i] ← down
                        ∀ j wait FLAG[j] = down
                        if Y = i then return
                             else wait Y = ⊥
                                  goto *
```

```
unlock(i) :=
    Y ← ⊥
    FLAG[i] ← down
    return
```

Listing 2.6: Lamport's Fast MUTEX Algorithm.

The logic for lock(i) is as follows:

1. Process p_i sets FLAG[i] to up and writes its identity to X.

2. It checks Y. If Y$\neq\perp$, it knows another process is active. It resets its flag to down, waits for Y to become $\perp$, and then restarts the protocol.

3. If Y$=\perp$, it sets Y to i. It then checks if X is still i. If X$= i$, the process has secured the "fast path" and enters the CS immediately. This takes only five accesses in contention-free scenarios.

4. If X$\neq i$ (meaning someone else overwrote X), the process must take the "slow path." It resets its flag, waits for all other processes to lower their flags, and then performs a final check on Y. If Y is still i, it enters the CS; otherwise, it waits for Y to be cleared and restarts.

The unlock(i) procedure is simple: p_i resets Y to $\perp$ and then lowers its FLAG[i].

Demonstration 2.4.1 – MUTEX proof for Lamport's Fast MUTEX Algorithm

Mutual exclusion ensures that if process p_i is in the CS, no other process p_j can be there simultaneously. The proof considers the two ways p_i can enter the CS:

- If p_i enters because it read X = i, it must have written Y = i after checking that Y was $\perp$. For p_j to enter, it would also need to find Y at $\perp$. If p_j wrote to X after p_i did, p_i would not have read X = i. Therefore, p_j must have written X earlier. However, this means p_j will find Y = i (set by p_i) and be forced to wait for p_i to unlock.

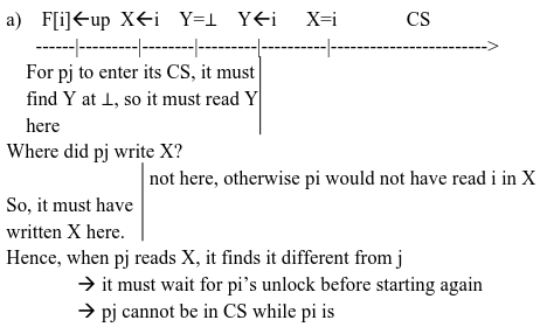


Figure 2.5: First case of the MUTEX proof for Lamport's Fast MUTEX Algorithm.

- If p_i enters via the slow path, it must have waited until all `FLAG[k]` were down. If p_j tries to enter, it must either finish its CS before p_i starts or be blocked by the fact that p_i has set `Y`$= i$. Since p_i only resets `Y` upon unlocking, p_j cannot progress past the gatekeeper check until p_i is finished.

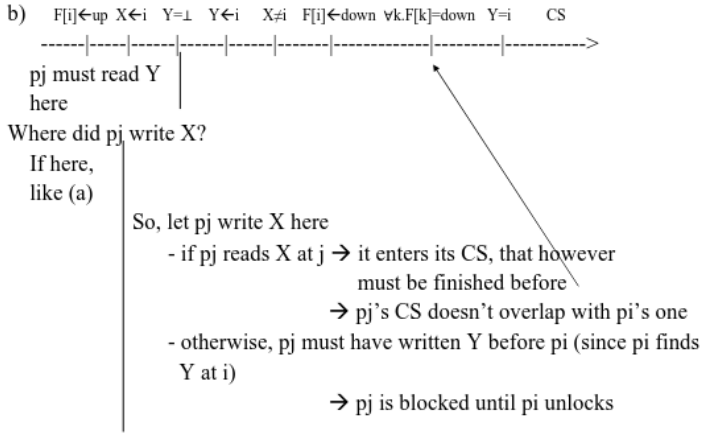


Figure 2.6: Second case of the MUTEX proof for Lamport's Fast MUTEX Algorithm.

Demonstration 2.4.2 – Deadlock freedom proof for Lamport's Fast MUTEX Algorithm

Deadlock freedom ensures that if a process invokes lock, at least one process eventually enters the CS. The proof analyzes the three points where a process might be blocked:

1. Second wait `Y`$=\perp$: If p_i is blocked here, it means it read a value in `Y` other than i. This implies another process p_h wrote to `Y` after p_i. By considering the "last" such process to write to `Y`, that process is guaranteed to eventually enter the CS or clear the way.
2. $\forall j$ `wait FLAG[j] = down`: This wait cannot block a process forever. If p_j is not trying to lock, its flag is down. If p_j is trying to lock but fails the `Y` or `X` checks, it eventually sets its flag to down. If p_j enters the CS, it will eventually unlock and set the flag to down.
3. First wait `Y` $=\perp$: If p_i is stuck here, it means some p_k wrote to `Y` first. As shown in the previous points, p_k will either enter the CS and eventually unlock (clearing `Y`) or it will be blocked elsewhere in a way that allows progress.

With atomic read/write registers, the trade-offs between different algorithms for n processes are summarized below in Table 2.1.

While Lamport's algorithm provides O(1) complexity and deadlock freedom, it is important to note that it can lead to starvation, where a specific process is bypassed indefinitely even though the system as a whole is making progress. Every deadlock-free algorithm can be transformed into a bounded bypass one, but this often comes at the cost of increased complexity.

Algorithm Type	Complexity (Contention-Free)	Liveness Property
Peterson (standard)	$O(n^2)$	Starvation-Freedom
Tournament-based	$O(\log n)$	Bounded Bypass
Lamport Fast MUTEX	$O(1)$	Deadlock-Freedom

Table 2.1: Summary of complexity for concurrent algorithms.

2.5 Turn Deadlock Freedom into Bounded Bypass

To understand the progression of mutual exclusion algorithms, we must examine how a protocol that guarantees only basic deadlock freedom can be elevated to provide a stronger progress condition: bounded bypass. The goal of this transformation is to take any deadlock-free protocol (denoted here as DLF) and wrap it in a structure that enforces fairness. This is achieved through the Round Robin algorithm. The name "Round Robin" itself finds its origins in the medieval French custom of signing petitions in a circular fashion—Ruban Rond or "round ribbon"—to obscure the identity of the leader or initiator, ensuring all participants shared equal standing. In the context of concurrent systems, it implies a circular, fair distribution of access.

The algorithm utilizes an array of flags, `FLAG[i]`, initialized to down for all processes, and a shared register `TURN`, which can be initialized to any process `ID`.

```
Initialize FLAG[i] to down (∀ i) and TURN to any proc.id.
```

```
lock(i) :=
    FLAG[i] ← up
    wait (TURN = i OR
          FLAG[TURN] = down)
    DLF.lock(i)
    return
```

```
unlock(i) :=
    FLAG[i] ← down
    if FLAG[TURN] = down then
        TURN ← (TURN+1) mod n
    DLF.unlock(i)
    return
```

Listing 2.7: Round Robin algorithm to turn a deadlock free protocol into a bounded bypass protocol for MUTEX.

For the entry protocol `lock(i)`, the process p_i first signals its intent by setting `FLAG[i]` to up. It then enters a busy-wait period where it stays suspended until one of two conditions is met: either it is its turn or the process whose turn it currently is is not interested in the critical section. Only after passing this "gate" does the process invoke the underlying deadlock-free locking mechanism.

The exit protocol `unlock(i)` mirrors this structure. The process first releases the underlying lock via `DLF.unlock(i)`. Then, it lowers its own flag and, if the current turn belongs to it, it increments the `TURN` variable to the next process in a circular fashion using the modulo operator.

The proof of deadlock freedom for the Round Robin algorithm relies on the fact that if at least one process invokes `RR.lock`, at least one process will eventually enter the critical section. Since we know the underlying DLF is deadlock-free, we simply need to prove that at least one process will eventually exit the initial wait statement to invoke `DLF.lock`.

If we assume `TURN=k`, and process p_k has invoked the lock, it will find that `TURN = k` is true and immediately proceed to `DLF.lock`. If p_k has not invoked the lock, its `FLAG[k]` will be down. In this case, any other process p_i that has invoked the lock will find that `FLAG[TURN] = down` is true and will likewise proceed to the underlying lock. Thus, the system as a whole cannot get stuck at the wrapper level.

To prove the bounded bypass property, we must quantify how long a process might have to wait.

Lemma 2.5.1 – Bounded bypass proof for Round Robin algorithm for MUTEX

If it is currently p_i's turn (`TURN = i`) and p_i is interested (`FLAG[i] = up`), it will enter the critical section in at most $(n-1)$ iterations of other processes.

Demonstration. This is supported by three key observations. First, `TURN` only changes when the process currently holding the turn sets its flag to down, which only happens after it finishes its critical section. Second, if p_i has its flag up, it is either already in the critical section or

is currently competing within the `DLF.lock` primitive. Third, any process p_j that invokes the lock after p_i has already set its flag will be blocked at the initial wait statement because `TURN = i` and `FLAG[i] = up`.
Consequently, we can define a set `Y` of processes currently competing for the critical section (those suspended on `DLF.lock`). Once p_i sets its flag, this set `Y` cannot grow. Because the underlying DLF is deadlock-free, some process p_y in `Y` must eventually win. If that process is p_i, the lemma is satisfied. If it is another process, that process enters the critical section, exits, and because `TURN` remains i, that process cannot re-enter the set `Y`. Thus, `Y` shrinks by one with each winner until p_i is eventually the only one left to win. The worst-case scenario is when `Y` contains all n processes, requiring $n-1$ other processes to finish before p_i.

Lemma 2.5.2 – Bounded bypass proof for Round Robin algorithm for MUTEX

If p_i is interested, `TURN` will be set to i in at most $(n-1)^2$ iterations.

Demonstration. If `TURN` is not already i, we know by the deadlock freedom of the Round Robin wrapper that at least one process will eventually unlock. If the process whose turn it is (p_{TURN}) is idle, `TURN` increments immediately. If p_{TURN} is active, we know from Lemma 2.5 that it will win and eventually increment `TURN` in at most $(n-1)$ iterations. Since there are at most $(n-1)$ processes that could hold the turn before it reaches i, the total iterations are $(n-1) \times (n-1) = (n-1)^2$.

The combined logic of these lemmas allows us to establish the final bound for the algorithm. Bounded bypass guarantees that if a process p_i invokes the lock, it will enter the critical section in at most $n(n-1)$ iterations.

The calculation is a summation of the two stages of waiting. First, p_i sets its flag. According to Lemma 2.5, it may take up to $(n-1)^2$ iterations for the `TURN` variable to cycle around to i. Once `TURN` is equal to i, Lemma 2.5 dictates that p_i will enter the critical section in at most $(n-1)$ further iterations. Adding these together $(n-1)^2 + (n-1)$ results in $n(n-1)$. This provides a strict upper bound on the number of times p_i can be bypassed by other processes, successfully transforming a deadlock-free protocol into a bounded bypass one.

2.6 Mutual Exclusion and Specialized Hardware

2.6.1 Specialized Hardware Primitives for Mutual Exclusion

While atomic read/write registers offer a foundational computational model for concurrent systems, they are often too basic for complex synchronization needs. To strengthen this model, most modern operating systems and shared memory multiprocessors provide specialized hardware primitives. These are built-in, atomic operations implemented directly in the hardware, specifically designed to address synchronization issues more efficiently than solutions based on simple read/write operations.

The most common of these hardware primitives include:

- *Test&set:* An atomic operation that reads a boolean register and writes a new value to it.
- *Swap:* An atomic operation that exchanges the contents of a general register with a given value.
- *Fetch&add:* An atomic operation that reads an integer register and increases its value by a specified amount.
- *Compare&swap:* An atomic operation that compares the current value of a register with an expected "old" value; if they match, it updates the register to a "new" value and returns a boolean indicating success.

Mutual Exclusion with Test&set

The Test&set primitive is a fundamental tool for implementing locks. When applied to a boolean register `X`, it atomically performs two steps: it stores the current value of `X` in a temporary variable

and then sets X to 1. The primitive then returns the original value stored in the temporary variable. Because this entire sequence is atomic, no other process can intervene between the read and the write.

```
1 X.test&set() :=
2     tmp ← X
3     X ← 1
4     return tmp
```

Listing 2.8: Test&set.

A simple mutual exclusion protocol can be established using this primitive. To acquire a lock, a process wait until X.test&set() returns 0.

```
1 Initialize X at 0
```

```
1 lock(i) :=
2     wait X.test&set() = 0
3     return
```

```
1 unlock(i) :=
2     X ← 0
3     return
```

Listing 2.9: MUTEX with Test&set.

The first process to call this when X=0 will receive 0 and simultaneously set X to 1, effectively "locking" the resource and causing subsequent calls by other processes to return 1. To unlock the resource, the process simply sets X back to 0.

Mutual Exclusion with Swap

The Swap primitive serves as a generalization of Test&set. It atomically exchanges the value of a general register X with a provided value v, returning the old value of X.

```
1 X.swap(v) :=
2     tmp ← X
3     X ← v
4     return tmp
```

Listing 2.10: Swap.

This primitive is highly versatile because the previously discussed Test&set protocol can be directly adapted to use it. Specifically, the operation X.test&set() is functionally equivalent to X.swap(1). By swapping in the value 1 and checking if the returned value was 0, a process can determine if it has successfully acquired the lock.

Mutual Exclusion with Compare&swap

Compare&swap takes two parameters, old and new, and checks if the shared register X currently holds the value old. If the condition X=old is met, X is updated to new and the operation returns true. Otherwise, the register remains unchanged, and the operation returns false.

```
1 X.compare&swap(old, new) :=
2     if X = old then X ← new
3                     return true
4     return false
```

Listing 2.11: Compare&swap.

To implement mutual exclusion with Compare&swap, the lock() operation involves waiting until X.compare&swap(0, 1) returns true. This ensures that only the process that finds X at 0 can change it to 1 and proceed. The unlock() operation is a simple assignment of 0 to X.

```
Initialize X at 0
```

```
lock() :=
    wait X.compare&swap(0,1) = true
    return
```

```
unlock(i) :=
     X ← 0
    return
```

Listing 2.12: MUTEX with Compare&swap.

While Compare&swap is powerful and can prevent many synchronization errors, protocols using only Test&set, Swap, or Compare&swap can suffer from starvation, where a process is indefinitely delayed because other processes keep winning the race to the lock.

Mutual Exclusion with Fetch&add

To address the issue of starvation and promote better liveness properties, a Round Robin approach can be implemented using the Fetch&add primitive. This primitive atomically adds a value `v` to an integer register `X` and returns the original value.

```
X.fetch&add(v) :=
    tmp ← X
    X ← X+v
    return tmp
```

Listing 2.13: Fetch&add.

A robust mutual exclusion protocol using Fetch&add employs a "ticket" system, similar to how customers are served in a bakery.

```
Initialize TICKET and NEXT at 0
```

```
lock() :=
    my_tick ← TICKET.fetch&add(1)
    wait my_tick = NEXT
    return
```

```
unlock() :=
    NEXT ← NEXT+1
    return
```

Listing 2.14: MUTEX with Fetch&add.

A process first obtains its unique ticket number by calling `my_tick` ← `TICKET.Fetch&add(1)`. It then waits until its ticket matches the current turn. Upon exiting the critical section, the process increments the turn counter to allow the next ticket holder to proceed.

This ticket-based algorithm ensures starvation-freedom and provides a bounded bypass of $n - 1$. This means that once a process has its ticket, it will enter the critical section after at most all other $n - 1$ processes that already had (or might concurrently obtain) their tickets have finished their turns.

2.6.2 Safe Registers

A critical question in concurrent systems is whether mutual exclusion can be enforced without hardware-level atomicity. *Safe Register* are the weakest type of register that remains useful for concurrency: unlike atomic registers, safe registers have much weaker guarantees:

- *MRSW (Multi-Reader Single-Writer) Safe Register:* If a `READ` operation does not overlap with a `WRITE` operation, it returns the value currently stored in the register. However, if a `READ` overlaps with a `WRITE`, the `READ` may return any value from the register's domain, even one that was never written.

- *MRMW (Multi-Reader Multi-Writer) Safe Register:* This behaves like an `MRSW` safe register as long as `WRITE` operations do not overlap. If `WRITE` operations overlap, the register's contents can become arbitrary (any value in its domain), and subsequent `READ`s may return any value until a non-overlapping `WRITE` restores a known state.

Lamport's Bakery Algorithm (1974)

The Bakery Algorithm, designed by Leslie Lamport, is the first algorithm created to solve the mutual exclusion problem using only these non-atomic, MRSW safe registers. The core idea is that every process wishing to enter the critical section receives a ticket. However, because the registers are not atomic, two processes might read the same current maximum ticket value and thus assign themselves identical tickets. To resolve these ties and create a total ordering, the algorithm uses a pair $\langle ticket, i \rangle$, where i is the process ID.

A total order is defined lexicographically: $\langle x, i \rangle < \langle y, j \rangle$ if $x < y$, or if $x = y$ and $i < j$.

```
Initialize FLAG[i] to down and MY_TURN[i] to 0, for all i
```

```
lock(i) :=
    FLAG[i] ← up
    MY_TURN[i] ← max{MY_TURN[1...,MY_TURN[n]}+1
    FLAG[i] ← down
    forall j ≠ i
        wait FLAG[j] = down
        wait (MY_TURN[j] = 0 OR
              ⟨MY_TURN[i],i⟩ < ⟨MY_TURN[j],j⟩)
```

```
unlock(i) :=
    MY_TURN[i] ← 0
```

Listing 2.15: Bakery algorithm (Lamport 1974) with Safe Registers.

Each process p_i manages two safe registers: `FLAG[i]` (initialized to `down`) and `MY_TURN[i]` (initialized to 0).

The `lock(i)` procedure consists of two main stages:

- *The Doorway:* The process raises its `FLAG[i]` to `up` and calculates its ticket. It then lowers `FLAG[i]` back to down.
- *The Bakery:* The process waits for every other process j. First, it waits until p_j is not in its doorway. Then, it waits until its own ticket is the smallest among active contenders.

In the `unlock(i)` procedure, the process simply resets its ticket to 0.

Lemma 2.6.1 – Bakery algorithm (Lamport 1974) with Safe Registers

If process p_i enters the bakery before p_j enters the doorway, then `MY_TURN[i]<MY_TURN[j]`.

Demonstration. Let t be the value p_i assigned to `MY_TURN[i]`. When p_j later computes its ticket in the doorway, it will read t from `MY_TURN[i]` (with no overlapping write). Therefore, p_j's ticket must be at least $t + 1$.

Lemma 2.6.2 – Bakery algorithm (Lamport 1974) with Safe Registers

If p_i is in the critical section (CS) and p_j is in either the doorway or the bakery, then $\langle MY\_TURN[i], i \rangle < \langle MY\_TURN[j], j \rangle$.

Demonstration. If p_i is in the CS, it has already passed its first wait for j (reading `FLAG[j] = down`). This read could have happened before p_j entered the doorway (in which case Lemma 2.6.2 applies) or after p_j left the doorway (meaning p_j is already in the bakery with a set ticket). In the latter case, p_i would have successfully evaluated the second wait condition, confirming its ticket pair was smaller.

Demonstration 2.6.1 – Properties of the Bakery Algorithm (Lamport 1974)

- *Mutual Exclusion:* This is proven by contradiction. If p_i and p_j were both in the CS, Lemma 2.6.2 would imply both $\langle MY\_TURN[i], i \rangle < \langle MY\_TURN[j], j \rangle$ and $\langle MY\_TURN[j], j \rangle < \langle MY\_TURN[i], i \rangle$, which is impossible.

- *Deadlock Freedom:* If a set of processes Q is blocked in the bakery, the process with the minimum ticket $\langle MY\_TURN[i], i\rangle$ will eventually succeed in its second wait. All other processes either have a `MY_TURN` of 0 (not competing) or a larger ticket pair, allowing p_i to proceed.
- *Bounded Bypass (Bound $n-1$):* If p_i and p_j are competing and p_j wins, once p_j exits and re-enters the protocol, it will obtain a new ticket. By Lemma 2.6.2, this new ticket will be strictly greater than p_i's existing ticket. Consequently, p_j cannot bypass p_i a second time. In the worst case, p_i may have to wait for all other $n-1$ processes to enter the CS once, but no more.

2.6.3 Aravind's Algorithm

In the study of mutual exclusion, a significant challenge arises with classic solutions like Lamport's Bakery algorithm: the requirement for unbounded registers. In such algorithms, every invocation of a lock potentially increments a counter, leading to a register domain that spans all natural numbers. While theoretically sound, this presents a practical implementation problem for real-world systems with finite memory. To address this, Aravind's algorithm (2011) introduces a mechanism that ensures mutual exclusion using registers with a bounded domain.

The primary goal of Aravind's construction is to constrain the range of values used for synchronization. In this model, each process p_i utilizes three shared registers:

- `FLAG[i]`: A binary Multi-Reader Single-Writer (MRSW) register, which indicates if a process is currently interested in entering the critical section (CS). It is initialized to "down".
- `STAGE[i]`: Another binary MRSW register used to coordinate the entry phase. It is initialized to 0.
- `DATE[i]`: A Multi-Reader Multi-Writer (MRMW) register that acts as a bounded timestamp, ranging from 1 to $2n$ (where n is the total number of processes). Initially, `DATE[i]` is set to the process index i.

```
For all i, initialize
    FLAG[i] to down
    STAGE[i] to 0
    DATE[i] to i
```

```
lock(i) :=
    FLAG[i] ← up
    repeat
        STAGE[i] ← 0
        wait (∀j≠i FLAG[j] = down OR
                   DATE[i] < DATE[j])
        STAGE[i] ← 1
    until ∀j≠i STAGE[j] = 0
```

```
unlock(i) :=
    tmp ← max_j{DATE[j]}+1
    if tmp ≥ 2n
        then ∀j DATE[j] ← j
        else DATE[i] ← tmp
    STAGE[i] ← 0
    FLAG[i] ← down
```

Listing 2.16: Aravind's algorithm.

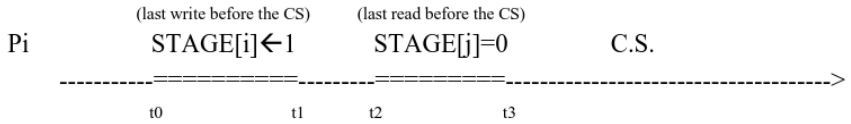
The entry protocol, or `lock(i)` routine, begins by raising `FLAG[i]` to "up." The process then enters a loop where it resets `STAGE[i]` to 0 and waits until every other process p_j either has its flag down or possesses a "date" (timestamp) greater than p_i's own. Once this condition is met, p_i sets `STAGE[i]` to 1. This loop repeats until p_i observes that all other processes have their `STAGE` registers at 0.

The exit protocol, `unlock(i)`, is where the boundedness is managed. The process calculates a temporary value, `tmp`, which is the current maximum date across all processes plus one. If this value reaches the bound of $2n$, a global reset is triggered, and all processes have their `DATE` registers reset to their respective indices `j`. If the bound is not reached, p_i simply updates its own `DATE` to the new `tmp` value. Finally, it resets its `STAGE` and `FLAG` to their initial states.

Theorem 2.6.1 – MUTEX proof for Aravind's algorithm

If p_i is in the critical section, p_j cannot be there simultaneously.

Demonstration. This is proved by contradiction through a temporal analysis of the STAGE register updates.



Consider the timeline leading to p_i entering the CS. At time t_0, p_i writes 1 to STAGE[i]. At t_1, it performs its last read of STAGE[j] and finds it to be 0. If we assume p_j also enters the CS, it must have also written STAGE[j]←1 and read STAGE[i]=0. However, because p_i has already written 1 to its stage register before reading p_j's, and p_j must do the same, their operations must overlap in a way that at least one process would see the other's STAGE as 1. This effectively blocks the concurrent entry.

Corollary. A useful corollary of this protocol is that the DATE registers are never written concurrently; because the updates occur within the unlock phase—which is protected by the mutual exclusion of the CS—only one process modifies these registers at any given time.

Lemma 2.6.3 – Bounded DATE register into Aravind's algorithm

A reset of the DATE values occurs exactly every n entries into the critical section.

Demonstration. This is because each entry into the CS increments the maximum date by one, and since the starting maximum is n and the limit is $2n$, it takes n steps to trigger the reset condition ($tmp \geq 2n$).

Lemma 2.6.4 – Bounded DATE register into Aravind's algorithm

During any single invocation of the lock routine by a process p_i, there can be at most one reset of the DATE registers.
This second lemma is vital for proving that the algorithm is starvation-free.

Demonstration. If a reset occurs while p_i is waiting, p_i might see other processes leapfrog it. However, because a second reset cannot occur before p_i enters, its progress is eventually guaranteed. In the worst-case scenario—where all n processes invoke the lock simultaneously and p_i has the highest index—all other $n-1$ processes might enter the CS before p_i, trigger a reset in their unlock phases, and potentially allow some processes to enter again before p_i finally gains access.

Theorem 2.6.2 – Bounded bypass proof for Aravind's algorithm

The algorithm satisfies the bounded bypass property with a bound of $2n-2$. This means a process can be bypassed by others at most $2n-2$ times before it is its turn to enter.

Demonstration. This bound is derived from the reset frequency: a process p_n might have to wait for $n-1$ processes to finish their CS entries to trigger a reset, and potentially another $n-1$ entries after the reset before its own DATE becomes the "oldest" in the system.

An improvement to Aravind's algorithm exists to tighten this bound to $n-1$. This revised version modifies the unlock phase. Instead of a simple increment and occasional global reset, the exiting process p_i proactively decrements the DATE of any process j that has a date greater than its own. It then sets its own DATE to n.

```
1 unlock(i) :=
2     ∀j≠i if DATE[j] > DATE[i] then DATE[j] ← DATE[j]-1
3     DATE[i] ← n
4     STAGE[i] ← 0
5     FLAG[i] ← down
```

Listing 2.17: Improvement of Aravind's algorithm.

Because the lock protocol remains the same, this improved version still satisfies mutual exclusion. However, by constantly "shifting" the dates of waiting processes downward, it ensures a stricter FIFO-like (First-In-First-Out) order, reducing the maximum number of times a process can be bypassed.

Chapter 3

Concurrent Objects

A *concurrent object* is defined as an entity that possesses a hidden internal implementation and a visible interface. This interface consists of a finite set of operations and a specification that describes its correct behaviors, typically in a sequential manner. When such an object is accessed by multiple processes simultaneously, it is termed a concurrent object.

3.1 Semaphores

The *semaphore* is a fundamental shared synchronization object used to manage concurrency. It is essentially a shared counter, `S`, accessed through two atomic primitives: `up` and `down`. A semaphore is initialized to a non-negative value `s0` and must always remain greater than or equal to zero. The `up` operation atomically increases `S` by 1, while the `down` operation decreases it by 1, provided the value is not zero. If `S` is zero, the process invoking `down` is blocked and must wait until another process performs an `up`. This mechanism is primarily used to prevent busy waiting by suspending processes that cannot immediately proceed. The relationship between the operations and the counter is maintained by the invariant $S = s0 + \#(S.up) - \#(S.down)$, where # denotes the number of completed operations.

In an ideal conceptual model, a semaphore consists of two underlying components: a counter that can become negative and a data structure, typically a FIFO queue, to store suspended processes. If the implementation counter is non-negative, it represents the actual value of the semaphore; if it is negative, its absolute value indicates the number of processes currently blocked in the queue.

```
1 S.down() :=
2     S.counter--
3     if S.counter < 0 then
4         enter into S.queue
5         SUSPEND
6     return
```

```
1 S.up() :=
2     S.counter++
3     if S.counter ≤ 0 then
4         activate a proc from S.queue
5     return
```

Listing 3.1: Ideal semaphore implementation.

A *strong semaphore* utilizes a FIFO policy for unblocking, whereas a *weak semaphore* one does not guarantee this order. A *binary semaphore* is restricted to values of 0 or 1, meaning even the `up` operation can be blocking if the semaphore is already at 1.

The actual implementation of these primitives requires hardware support to ensure atomicity. This is achieved using a test&set register—an atomic hardware instruction that sets a value while returning the old one—and by disabling interrupts to prevent the scheduler from preempting a process mid-operation.

```
Let t be a test&set register initialized at 0
```

```
S.down() :=
    Disable interrupts
    wait S.t.test&set() = 0
    S.counter--
    if S.counter < 0 then
        enter into S.queue
        S.t ← 0
        Enable interrupts
        SUSPEND
    else S.t ← 0
        Enable interrupts
    return
```

```
S.up() :=
    Disable interrupts
    wait S.t.test&set() = 0
    S.count++
    if S.count ≤ 0 then
        activate a proc from S.queue
    S.t ← 0
    Enable interrupts
    return
```

Listing 3.2: Actual semaphore implementation.

In the implementation code, a process must first acquire a local lock (using test&set) and disable interrupts before modifying the counter or the queue. Once the operation is complete, it resets the lock and re-enables interrupts. If a `down` operation results in a negative counter, the process is added to the queue and enters a `SUSPEND` state, relinquishing the processor until a corresponding `up` operation activates it.

3.1.1 The Producer-Consumer Problem

The *Producer-Consumer problem* involves a shared FIFO buffer of size k. The internal representation of the buffer uses an array `BUF[0...k-1]` and two index variables, `IN` and `OUT`, which point to where items are inserted and removed circularly. In a single producer/consumer scenario, two semaphores are used: `FREE`, initialized at k, and `BUSY`, initialized at 0.

```
B.produce(v) :=
    FREE.down()
    BUF[IN] ← v
    IN ← (IN+1) mod k
    BUSY.up()
    return
```

```
B.consume() :=
    BUSY.down()
    tmp ← BUF[OUT]
    OUT ← (OUT+1) mod k
    FREE.up()
return tmp
```

Listing 3.3: (Single) Producer/Consumer.

The producer must perform a `FREE.down()` to ensure there is space before writing to `BUF[IN]` and incrementing `IN`. Conversely, the consumer performs a `BUSY.down()` to ensure there is data to read before accessing `BUF[OUT]` and incrementing `OUT`. This design ensures that the producer never overwrites unread data and the consumer never reads from an empty cell. However, reading or writing to the buffer can be computationally expensive, making mutual exclusion around the entire buffer a potential bottleneck.

When multiple producers or consumers are introduced, the variables `IN` and `OUT` must be protected from concurrent access to prevent two processes from using the same index. To allow for parallel access to different cells of the buffer, a more complex solution employs two arrays of atomic boolean registers, `BUSY` and `EMPTY`, to track the state of each individual cell. Two additional semaphores, `SP` and `SC` (both initialized to 1), are used to provide mutual exclusion specifically for the selection of the `IN` and `OUT` indices.

A "wrong" approach to this problem involves a producer taking an index `i = IN`, incrementing `IN` immediately, and then attempting to write.

```
1 B.produce(v) :=
2     FREE.down()
3     SP.down()
4     i ← IN
5     IN ← (i+1) mod k
6     SP.up()
7     BUF[i] ← v
8     BUSY.up()
9     return
```

```
1 B.consume() :=
2     BUSY.down()
3     SC.down()
4     o ← OUT
5     OUT ← (o+1) mod k
6     SC.up()
7     tmp ← BUF[o]
8     FREE.up()
9     return tmp
```

Listing 3.4: Wrong (Multiple) Producer/Consumer.

If a very slow consumer is processing the first item while multiple quick producers fill the rest of the buffer, the indices can loop back around. Without proper cell-level checks (like the `while` $\neq$ `EMPTY[IN]` loop), a fast producer might attempt to overwrite a cell that the slow consumer is still reading, even if the general `FREE` semaphore allows the producer to proceed.

```
1  B.produce(v) :=
2      FREE.down()
3      SP.down()
4      while ¬EMPTY[IN] do
5          IN ← (IN+1) mod k
6      i ← IN
7      EMPTY[i] ← ff
8      SP.up()
9      BUF[i] ← v
10     FULL[i] ← tt
11     BUSY.up()
12     return
```

```
1  B.consume() :=
2      BUSY.down()
3      SC.down()
4      while ¬FULL[OUT] do
5          OUT ← (OUT+1) mod k
6      o ← OUT
7      FULL[o] ← ff
8      SC.up()
9      tmp ← BUF[o]
10     EMPTY[o] ← tt
11     FREE.up()
12     return tmp
```

Listing 3.5: Correct (Multiple) Producer/Consumer.

3.1.2 The Readers/Writers Problem

The *Readers-Writers problem* generalizes mutual exclusion by allowing multiple "readers" to access a resource (like a file) simultaneously, while "writers" require exclusive access.

```
1 conc_read() :=
2     begin_read()
3     read()
4     end_read()
```

```
1 conc_write() :=
2     begin_write()
3     write()
4     end_write()
```

Listing 3.6: Read/write operations on the file for the readers/writers problem.

This leads to different priority schemes to manage access:

- *Weak Priority to Readers:* In this scheme, if a reader arrives while another reader is already active, it can enter even if a writer is waiting. A `GLOB_MUTEX` semaphore manages access to the file, but only the first reader to arrive and the last reader to leave interact with it. A separate `R_MUTEX` ensures that the reader count `R` is updated atomically.

```
GLOB_MUTEX and R_MUTEX semaphores init. at 1
R a shared register init. at 0
```

```
begin_read() :=
    R_MUTEX.down()
    R++
    if R = 1 then GLOB_MUTEX.down()
    R_MUTEX.up()
    return
```

```
end_read() :=
    R_MUTEX.down()
    R--
    if R = 0 then GLOB_MUTEX.up()
    R_MUTEX.up()
    return
```

```
begin_write() :=
    GLOB_MUTEX.down()
    return
```

```
end_write() :=
    GLOB_MUTEX.up()
    return
```

Listing 3.7: Weak Priority to Readers.

- *Strong Priority to Readers:* This is a refinement where, upon a writer's termination, any waiting readers are prioritized over waiting writers. This is implemented by adding a `W_MUTEX` that writers must acquire, effectively allowing readers to "queue up" and bypass writers more efficiently.

```
GLOB_MUTEX, R_MUTEX and W_MUTEX semaphores init. at 1
R a shared register init. at 0
```

```
begin_read() :=
    R_MUTEX.down()
    R++
    if R = 1 then GLOB_MUTEX.down()
    R_MUTEX.up()
    return
```

```
end_read() :=
    R_MUTEX.down()
    R--
    if R = 0 then GLOB_MUTEX.up()
    R_MUTEX.up()
    return
```

```
begin_write() :=
    W_MUTEX.down()
    GLOB_MUTEX.down()
    return
```

```
end_write() :=
    GLOB_MUTEX.up()
    W_MUTEX.up()
    return
```

Listing 3.8: Strong priority to Readers.

- *Weak Priority to Writers:* Here, the goal is to prevent writers from starving. If a writer is waiting, arriving readers are blocked. This is achieved using a `PRIO_MUTEX`. When a writer wants to access the file, it increments a writer count `W`; the first writer to arrive grabs the `PRIO_MUTEX`, which blocks any new readers from entering the system until all current writers have finished their work and the last writer releases the `PRIO_MUTEX`.

```
GLOB_MUTEX, PRIO_MUTEX, R_MUTEX and W_MUTEX semaphores init. at 1
R and W shared registers init. at 0
```

```
begin_read() :=
    PRIO_MUTEX.down()
    R_MUTEX.down()
    R++
    if R = 1 then GLOB_MUTEX.down()
    R_MUTEX.up()
    PRIO_MUTEX.up()
    return
```

```
end_read() :=
    R_MUTEX.down()
    R--
    if R = 0 then GLOB_MUTEX.up()
    R_MUTEX.up()
    return
```

```
begin_write() :=
    W_MUTEX.down()
    W++
    if W = 1 then PRIO_MUTEX.down()
    W_MUTEX.up()
    GLOB_MUTEX.down()
    return
```

```
end_write() :=
    GLOB_MUTEX.up()
    W_MUTEX.down()
    W--
    if W = 0 then PRIO_MUTEX.up()
    W_MUTEX.up()
    return
```

Listing 3.9: Weak priority to Writers.

3.2 Monitors

In the evolution of concurrent programming, semaphores represented a foundational step, yet their low-level nature often makes them difficult to use correctly in complex systems. Monitors were introduced as a higher-level linguistic construct to provide a more structured approach to defining concurrent objects. A *monitor* is an abstract data type that guarantees mutual exclusion by design: at most one process can be active inside the monitor at any given time. This encapsulation simplifies the development of concurrent systems by moving the responsibility of mutual exclusion from the programmer to the programming language or runtime environment.

While mutual exclusion is handled automatically by the monitor, internal synchronization between processes—such as waiting for a specific condition to become true—is managed through condition variables. A *condition* is an object that facilitates two primary operations: `wait` and `signal`. When a process invokes `C.wait()`, it suspends execution, joins the queue associated with condition `C`, and importantly, releases the mutual exclusion lock on the monitor so other processes can enter.

The `signal` operation behaves differently depending on the specific semantics of the monitor. If no processes are waiting on the condition, `signal` typically has no effect. However, if processes are suspended, the behavior branches into two main schools of thought. Under *Hoare semantics*, the signaling process immediately suspends itself and passes the monitor lock to the first process in the condition's queue. This reactivated process has priority to re-enter or continue, and the signaling process only resumes once the monitor becomes free again. Conversely, *Mesa semantics* allow the signaling process to complete its current task or reach a suspension point before any signaled process is allowed to re-enter. In this model, the signal is more of a hint that the condition might be true, often requiring the signaled process to re-check the condition upon waking.

3.2.1 Rendezvous through monitors

A practical application of monitors is the implementation of a *Rendezvous* or *Barrier*. This concurrent object is associated with m control points, representing m distinct processes. The goal is to ensure that no process can pass its control point until every other involved process has reached theirs.

```
monitor RNDV :=
    cnt ∈ {0,...,m} init at 0

    condition B

    operation barrier() :=
        cnt++
        if cnt < m then B.wait()
                   else cnt ← 0
        B.signal()
        return
```

Listing 3.10: Rendezvous through monitors.

In a monitor-based implementation, a counter `cnt` tracks how many processes have arrived. Each process calls the `barrier()` operation. If the counter has not yet reached `m`, the process increments `cnt` and executes `B.wait()`, suspending itself. The last process to arrive (where `cnt` reaches `m`) does not wait; instead, it resets the counter to zero and calls `B.signal()` to wake the waiting processes. This pattern ensures that all processes are synchronized at a specific logical point before proceeding.

To understand the low-level mechanics of monitors, we can examine how they are implemented using semaphores. This requires managing several components: a `MUTEX` semaphore (initialized to 1) to ensure the monitor's primary mutual exclusion, and for each condition `C`, a semaphore $\texttt{SEM}_C$ (initialized to 0) along with a counter N_C to track suspended processes. To handle the priority requirements of Hoare semantics, an additional `PRIO` semaphore and an N_{PR} counter are used to manage processes that have performed a signal and are waiting to re-gain control of the monitor.

Every operation within the monitor is wrapped in a protocol. It begins with `MUTEX.down()`. Upon completion, the operation checks if any signaling processes are waiting (N_{PR} `> 0`); if so, it releases one via `PRIO.up()`, otherwise, it releases the monitor via `MUTEX.up()`.

```
if N_PR > 0 then PRIO.up() else MUTEX.up()
```

The `wait` and `signal` primitives are implemented as follows:

- `C.wait()`: The process increments the count of waiting processes (N_C), releases the monitor (prioritizing signaling processes if they exist), and then blocks on $\texttt{SEM}_C$`.down()`. Once woken, it decrements N_C.

```
C.wait() :=
    N_C++
    if N_PR > 0 then PRIO.up() else MUTEX.up()
    SEM_C.down()
    N_C--
    return
```

- `C.signal()`: If there are processes waiting on the condition (N_C `> 0`), the signaling process increments the priority count (N_{PR}), wakes a waiting process via $\texttt{SEM}_C$`.up()`, and then blocks itself on `PRIO.down()` to allow the signaled process to run. When it eventually resumes, it decrements N_{PR}.

```
C.signal() :=
    if N_C > 0 then N_PR++
                    SEM_C.up()
                    PRIO.down()
                    N_PR--
                    return
```

3.2.2 Monitors for Readers/Writers

The Readers/Writers problem can be elegantly solved using monitors by maintaining four state variables: Active Readers (`AR`), Waiting Readers (`WR`), Active Writers (`AW`), and Waiting Writers (`WW`).

```
monitor RW_READERS :=
    AR, WR, AW, WW init at 0
    condition CR, CW
```

```
operation begin_read() :=
    WR++
    if AW≠0 then CR.wait()
                 CR.signal()
    AR++
    WR--
```

```
operation end_read() :=
    AR--
    if AR+WR=0 then CW.signal()
```

```
operation begin_write() :=
    if (AR+WR≠0 OR AW≠0) then
        CW.wait()
    AW++
```

```
operation end_write() :=
    AW--
    if WR > 0 then
        CR.signal()
    else CW.signal()
```

Listing 3.11: Monitors Strong Priority to Readers.

In the "Strong Priority to Readers" implementation, a reader can enter as long as no writer is currently active (`AW == 0`). If a writer is active, the reader increments `WR` and waits on `CR`. When a reader finishes (`end_read`), if it is the last reader and no more readers are waiting, it signals a writer. This solution can lead to writer starvation, as a continuous stream of readers can keep the monitor in a state where `AW` remains zero while `AR` remains positive.

```
monitor RW_WRITERS :=
    AR, WR, AW, WW init at 0
    condition CR, CW
```

```
operation begin_read() :=
    if WW+AW≠0 then CR.wait()
                    CR.signal()
    AR++
```

```
operation end_read() :=
    AR--
    if AR=0 then CW.signal()
```

```
operation begin_write() :=
    WW++
    if AR+AW≠0 then CW.wait()
    AW++
    WW--
```

```
operation end_write() :=
    AW--
    if WW > 0 then CW.signal()
              else CR.signal()
```

Listing 3.12: Monitors Strong Priority to Writers.

Conversely, the "Strong Priority to Writers" version blocks new readers if there are any writers waiting (`WW + AW != 0`). In `begin_write`, a writer increments `WW` and waits if any readers or writers are active. In `end_write`, the finishing writer checks if there are other writers waiting and signals them first. Only if no writers remain does it signal the readers. This version risks starving the readers.

```
monitor RW_FAIR :=
    AR, WR, AW, WW init at 0
condition CR, CW
```

```
operation begin_read() :=
    WR++
    if WW+AW≠0 then CR.wait()
                    CR.signal()
    AR++
    WR--
```

```
operation end_read() :=
    AR--
    if AR=0 then CW.signal()
```

```
begin_write() :=
    WW++
    if AR+AW≠0 then CW.wait()
    AW++
    WW--
```

```
operation end_write() :=
    AW--
    if WR > 0 then CR.signal()
              else CW.signal()
```

Listing 3.13: Monitors fair solution to Readers/Writers.

A fair solution seeks to balance these interests. It dictates that during a read, new readers must wait if writers are already in the queue (`WW + AW != 0`). However, once a write operation finishes, it signals waiting readers to allow them to proceed. This prevents either group from monopolizing the resource indefinitely.

3.2.3 The Dining Philosophers Problem

The *Dining Philosophers problem*, introduced by Dijkstra in 1965, illustrates the challenges of resource allocation and deadlock. N philosophers sit at a circular table with N chopsticks; each needs two chopsticks to eat.

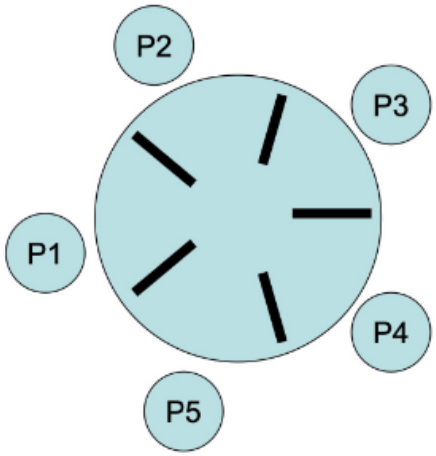


Figure 3.1: Dining Philosophers

A naive semaphore solution, where each philosopher grabs the left chopstick and then the right, is susceptible to deadlock: if every philosopher picks up their left chopstick simultaneously, no one can pick up their right, and the system freezes.

```
semaphore chopstick[5] initialized to 1
```

```
Philosopher(i) :=
    while(1) do
        chopstick[i].down()
        chopstick[(i+1)%N].down()
        // eat
        chopstick[(i+1)%N].up()
        chopstick[i].up()
```

Listing 3.14: Deadlock solution for Dining Philosophers Problem.

Several strategies exist to break this symmetry and ensure a deadlock-free execution:

1. *Resource Hierarchy:* Assign a unique number to each fork. Philosophers must always attempt to pick up the lower-numbered fork first. This forces the last philosopher to pick up the forks in the opposite order of the others, breaking the circular wait.

```
semaphore chopstick[5] initialized to 1
```

```
Philosopher(i) :=
    Repeat
    think;
    if (i < N-1) then
        fork[i].down();
        fork[i+1].down();
    else
        fork[0].down();
        fork[N-1].down();
    eat;
    fork[(i+1)%N].up();
    fork[i].up();
```

Listing 3.15: Resource Hierarchy solution for Dining Philosophers Problem.

2. *Parity-Based Selection:* Odd-numbered philosophers pick up the left fork then the right, while even-numbered philosophers pick up the right then the left. This prevents the circular dependency required for deadlock.

```
semaphore chopstick[5] initialized to 1
```

```
Philosopher(i) :=
    Repeat
        think;
        if (i % 2 == 0) then
            fork[i].down();
            fork[(i+1)%N].down();
        else
            fork[(i+1)%N].down();
            fork[i].down();
        eat;
        fork[(i+1)%N].up();
        fork[i].up();
```

Listing 3.16: Parity-Based Selection solution for Dining Philosophers Problem.

3. *Admission Control:* Use a "table" semaphore initialized to $N - 1$. By allowing only four philosophers at a table of five, at least one philosopher is guaranteed to have access to two chopsticks.

```
semaphores fork[N] all initialized at 1
semaphore table initialized at N-1

Philosopher(i) :=
    Repeat
        think;
        table.down();
        fork[i].down();
        fork[(i+1)%N].down();
        eat;
        fork[(i+1)%N].up();
        fork[i].up();
        table.up()
```

Listing 3.17: Admission Control solution for Dining Philosophers Problem.

4. *Monitor-Based State Testing:* This solution uses a monitor to track the state of each philosopher (thinking, hungry, or eating). A philosopher only moves to the eating state if both neighbors are not currently eating. This is managed via a `test(i)` function. If a philosopher cannot eat, they wait on a condition variable `self[i]`. When a neighbor finishes eating and calls `Putdown`, they execute test on their neighbors, signaling any who are hungry and now able to eat. This approach ensures that the "hungry" state is correctly transitioned only when the necessary resources are actually available.

```
monitor DP
    status state[N] all initialized at thinking;
    condition self[N];

    Pickup(i) :=
        state[i] = hungry;
        test(i);
        if (state[i] != eating) then self[i].wait;

    Putdown(i) :=
        state[i] = thinking;
        test((i+1)%N);
        test((i-1)%N);

    test(i) :=
        if (state[(i+1)%N] != eating && state[(i-1)%N] != eating
            && state[i] == hungry)
        then state[i] = eating;
             self[i].signal();
```

Listing 3.18: Monitor-Based State Testing solution for Dining Philosophers Problem.

Chapter 4

Software Transactional Memory

Software Transactional Memory (STM) is a high-level synchronization mechanism designed to simplify concurrent programming. By providing an abstraction similar to database transactions, STM allows developers to define blocks of code that must execute atomically without having to manually manage low-level synchronization primitives like locks.

An STM system groups together parts of the application code that must appear atomic to an external observer. Unlike traditional monitors, where atomicity is tied to the definition of an object, STM allows atomicity to be application-dependent. Furthermore, unlike database transactions which are often limited to SQL-like queries, STM transactions can consist of any arbitrary code accessing shared atomic objects.

A *transaction* is defined as an atomic unit of computation that accesses "t-objects" (atomic objects used within transactions). It is assumed that any transaction will successfully terminate if executed in isolation. A *program* in this context consists of multiple sequential processes that alternate between transactional and non-transactional code. The role of the *STM system* is to act as an online algorithm ensuring that all transactional code executes with the appearance of atomicity.

To maintain efficiency, STM systems typically employ *optimistic execution*, where multiple transactions run simultaneously. If two transactions conflict (e.g., both access the same object and at least one modifies it), the system must ensure they can still be totally ordered. If a correct ordering is impossible, a transaction is aborted and may be re-executed or the parent process notified. This introduces the possibility of unbounded delays, which is the "price" of synchronization.

Conceptually, a transaction involves three distinct phases:

1. *Read Phase:* Reading values from atomic registers.

2. *Local Computation:* Processing data within a local workspace.

3. *Write Phase:* Updating shared memory with new values.

Maintaining the consistency of shared memory is critical; if any inconsistency is detected during execution, the transaction must abort. Most implementations use a local working space to isolate changes until they are ready to be committed. When a shared register is first read, its value is copied locally. Subsequent reads use this local copy. Writes only modify the local copy and are only pushed to shared memory at the end of the transaction if the commit is successful.

The standard STM interface provides four operations:

- `begin`$_T$`()`: Initializes local control variables and timestamps.

- `X.read`$_T$`()`: Reads the implementation register `XX`. If it is the first read, it copies the value to the local workspace.

- `X.write`$_T$`(v)`: Updates the local copy of the register with value `v`.

- `try_to_commit`$_T$`()`: Validates the transaction's read set and determines if it can safely commit its changes to shared memory.

4.1 Opacity

A common approach to STM consistency is *opacity*, which requires that all committed transactions, as well as the read prefixes of all aborted transactions, appear to execute in a sequential order that respects their real-time occurrence. This prevents transactions from ever reading from an inconsistent global state.

Transactional Locking 2 (TL2) is a logical clock-based STM system that ensures opacity. It utilizes a global atomic `CLOCK` register. Every application register `X` is represented by an implementation pair `XX` containing the value (`XX.val`) and the date of its last update (`XX.date`).

In TL2, each transaction T maintains:

- `lc(XX)`: Local copies of implementation registers.
- `read_set(T)` and `write_set(T)`: Sets tracking accessed registers.
- `birthdate(T)`: The value of `CLOCK` at the start of the transaction.

Consistency is maintained by ensuring that for any read, the register's version date is less than the transaction's birthdate (`XX.date < birthdate(T)`). During $\texttt{try\_to\_commit}_T\texttt{()}$, the system locks all registers in the read and write sets, re-validates their dates against the birthdate, and—if valid—increments the global clock to provide a new timestamp for the written values. To prevent deadlocks during the locking phase, registers are always locked according to a predefined total order.

```
1  beginT() :=
2      read_set(T), write_set(T) ← ∅
3      birthdate(T) ← CLOCK+1
```

```
1  X.readT() :=
2      if lc(XX)≠⊥ then return lc(XX).val
3      lc(XX) ← XX
4      if lc(XX).date ≥ birthdate(T) then ABORT
5      read_set(T) ← read_set(T) ∪ {X}
6      return lc(XX).val
```

```
1  X.writeT(v) :=
2      if lc(XX)=⊥ then lc(XX) ← newloc
3      lc(XX).val ← v
4      write_set(T) ← write_set(T) ∪ {X}
```

```
1  try_to_commitT() :=
2      lock all read_set(T) ∪ write_set(T)
3      ∀ X ∈ read_set(T)
4          if XX.date ≥ birthdate(T)
5          then release all locks
6              ABORT
7      tmp ← CLOCK.fetch&add(1)+1
8      ∀ X ∈ write_set(T)
9          XX ← ⟨lc(XX).val, tmp⟩
10     release all locks
11     COMMIT
```

Listing 4.1: A Logical Clock based STM system.

4.2 Virtual World Consistency

Opacity is a strong condition that can be restrictive. *Virtual World Consistency (VWC)* is a more liberal property. While it still requires committed transactions to be linearizable, it only requires aborted transactions to be consistent with their own causal past (their "virtual world") rather than a single global history. The causal past of a transaction `T` includes any committed transaction `T'` that `T` has read from, along with all transactions that preceded `T'`.

Consider an example with four transactions: T_1, T_2, T_3, and T_4 (Figure 4.1). In an execution where T_1 and T_2 are sequential and T_3 is concurrent, opacity might force T_1 to abort if T_4 (an aborted transaction) reads an inconsistent value between T_2 and T_3. However, under VWC, T_1 can commit because T_4 only needs to be consistent relative to its specific causal dependencies (e.g., $T_2 < r_p ref(T_4)$), even if it cannot be globally ordered with T_3.

To implement VWC efficiently and avoid the bottleneck of a single global logical clock, a *vector clock-based STM* can be used. In this system, each of the m registers stores a vector clock `XX.depend[1...m]`. `XX.depend[X]` represents the sequence number of the current value of `X`, while `XX.depend[Y]` tracks the sequence number of register `Y` upon which the current value of `X` depends.

Processes maintain their own local vector clocks (`p_depend`), which are used to initialize a transaction's local dependency vector ($\texttt{t\_depend}_T$).

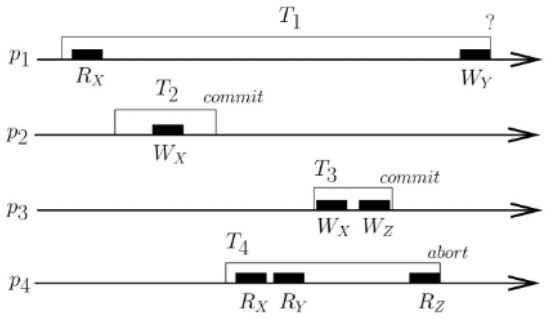


Figure 4.1: Virtual World Consistency example.

```
1  begin_T(i) :=
2      read_set(T), write_set(T) ← ∅
3      t_depend_T ← p_depend_i
4
5  X.read_T(i) :=
6      if lc(XX)=⊥ then
7          lc(XX) ← newloc
8          lc(XX) ← XX
9          read_set(T) ← read_set(T) ∪ {X}
10         t_depend_T[X] ← lc(XX).depend[X]
11         if ∃ Y ∈ read_set(T) s.t.
12             t_depend_T[Y] < lc(XX).depend[Y]
13         then ABORT
14         ∀ Y ∉ read_set(T) do
15             t_depend_T[Y] ← max{t_depend_T[Y],
16                                lc(XX).depend[Y]}
17     return lc(XX).val
```

```
1  X.write_T(i,v) :=
2      if lc(XX)=⊥ then lc(XX) ← newloc
3      lc(XX).val ← v
4      write_set(T) ← write_set(T) ∪ {X}
5
6  try_to_commit_T(i) :=
7      lock all read_set(T) ∪ write_set(T)
8      if ∃ Y ∈ read_set(T) s.t.
9          t_depend_T[Y] < YY.depend[Y]
10     then release all locks
11         ABORT
12     ∀ X ∈ write_set(T) do
13         t_depend_T[X] ← XX.depend[X]+1
14     ∀ X ∈ write_set(T) do
15         XX ← ⟨lc(XX).val, t_depend_T⟩
16     release all locks
17     p_depend_i ← t_depend_T
18     COMMIT
```

Listing 4.2: A Vector clock based STM system.

The consistency check during `X.read`$_T$`()` ensures that the value of `X` does not depend on a version of any register `Y` that is "newer" than what the transaction has already seen (`t_depend`$_T$`[Y] < lc(XX).depend[Y]`). If this check passes, `t_depend`$_T$ is updated to include the new dependencies from the register just read. This approach allows for higher concurrency by localizing consistency checks to relevant dependencies rather than a global timeline.

Chapter 5

Atomicity

This chapter explores the formal foundations of *atomicity*, a fundamental consistency condition in concurrent systems. As introduced in earlier contexts, atomicity allows us to judge the correctness of a concurrent implementation by relating it to a sequential execution. This formal perspective, often referred to as linearizability when applied to any concurrent object, provides a framework for reasoning about processes that access shared resources.

We consider a system composed of a set of n sequential processes, $p_1, \ldots p_n$, which access m concurrent objects, $X_1, \ldots, X_m$. Processes interact with these objects by invoking operations of the form X_i`.op(args)(ret)`. To model the duration and concurrency of these operations, each invocation by a process p_j is represented by two distinct events:

- An invocation event: `inv[`X_i`.op(args) by `p_j`]`
- A response event: `res[`X_i`.op(ret) to `p_j`]`

A *history* (or *trace*), denoted as $\widehat{H} = (H, <_H)$, consists of a set of these events H and a total order $<_H$ defined over them. This order must satisfy the basic requirement that every response event follows its corresponding invocation event. The semantics of the system or its individual objects are defined as the set of all possible valid traces they can produce.

Histories can be classified based on their structure:

- *Sequential History:* A history where events appear in strictly alternating pairs: $inv, res, inv, res, \ldots$ In such a history, every response is the immediate successor of its corresponding invocation. Because of this structure, a sequential history can be simplified and represented merely as a sequence of operations.
- *Complete vs. Partial:* A history is complete if every invocation event is eventually matched by a corresponding response event. If some invocations remain "hanging" without a return, the history is considered partial.

5.1 Linearizability

Linearizability is the gold standard for correctness in concurrent object design. A complete history $\widehat{H}$ is defined as linearizable if there exists a equivalent sequential history $\widehat{S}$ that satisfies three specific conditions:

1. *Object Semantics:* For every object X, the restriction of $\widehat{S}$ to X (denoted $\widehat{S}|_X$) must belong to the sequential specification (semantics) of that object.

$$\forall X \widehat{S}|_X \in semantics(X)$$

2. *Process Consistency:* For every process p, the order of operations in $\widehat{H}$ as seen by p must be identical to the order in $\widehat{S}$ (i.e., $\widehat{H}|p = \widehat{S}|p$).

$$\forall p \widehat{H}|_p = \widehat{S}|_p$$

3. *Real-Time Order Preservance:* If an operation op completes in $\widehat{H}$ before another operation op' begins (meaning $res[op] <_H inv[op']$), then op must also precede op' in the sequential history $\widehat{S}$ ($res[op] <_S inv[op']$).

$$\textit{if } res[op] <_H inv[op'], \textit{ then } res[op] <_S inv[op']$$

To formalize the third condition, we define a binary relation $\longrightarrow_H$ such that $(op, op') \in \longrightarrow_H$ (written $op \longrightarrow_H op'$) if and only if $res[op] <_H inv[op']$. Linearizability essentially requires that the sequential order $\longrightarrow_S$ is a superset of the real-time precedence order $\longrightarrow_H$ ($\longrightarrow_H \subseteq \longrightarrow_S$).

Example 5.1.1 – A Linearizable Queue

Consider a queue Q accessed by two processes. Process p_1 enqueues a and b (`Q.enq(a)` and `Q.enq(b)`), while process p_2 enqueues c and performs a dequeue (`Q.deq->a`).

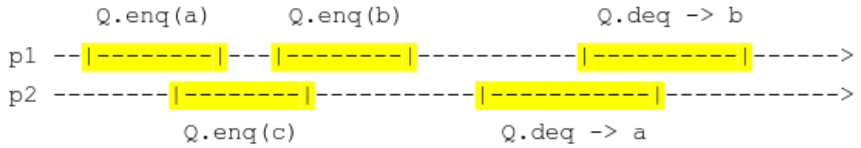


Figure 5.1: Linearizable Queue Timeline.

The physical timeline might show these operations overlapping. For instance, p_2's `enq(c)` might start after p_1's `enq(a)` has started but before it finishes. If p_2 eventually dequeues a, we must find a sequential order that makes sense. One such linearization is:

$$[Q.enq(a)] \rightarrow [Q.enq(b)] \rightarrow [Q.enq(c)] \rightarrow [Q.deq \rightarrow a] \rightarrow [Q.deq \rightarrow b].$$

This sequence satisfies the FIFO (First-In-First-Out) semantics of a queue and respects the order in which individual processes issued their commands.
However, certain histories are not linearizable. If p_2 dequeues a in a context where `enq(a)` clearly happened after another completed operation that remains in the queue, or if the real-time order is violated (e.g., p_2 dequeues a but `enq(a)` starts only after p_2's dequeue has already finished), the history cannot be linearized.

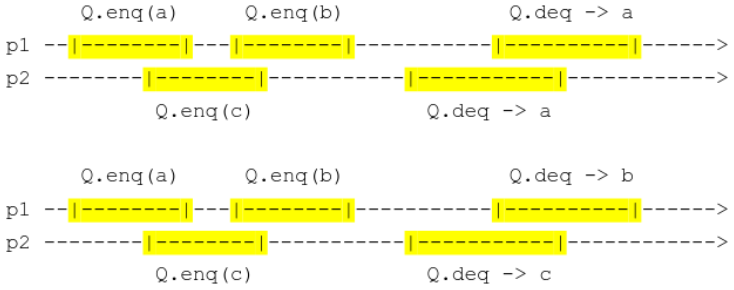


Figure 5.2: Non-Linearizable Queue Timeline.

5.1.1 Compositionality of Linearizability

Theorem 5.1.1 – Compositionality

One of the most powerful properties of linearizability is *compositionality* (also known as the *locality property* in the literature). This property states that a history $\widehat{H}$ is linearizable if and only if its restriction to every individual object X ($\widehat{H}|_X$) is linearizable.
This allows system designers to verify the correctness of each object in isolation, guaranteed that their combination will also be correct.

Demonstration. To prove this, we assume that for every object X, there is a linearization $\widehat{S}_X$. This $\widehat{S}_X$ defines a total order on operations for that specific object, which we call $\longrightarrow_X$. We then define a global relation $\longrightarrow$ as the union of the real-time order $\longrightarrow_H$ and all individual

object orders $\longrightarrow_X$:

$$\longrightarrow = \longrightarrow_H \cup \bigcup_X \longrightarrow_X$$

The core of the proof lies in showing that this relation $\longrightarrow$ is acyclic:

1. *No self-loops:* An operation cannot precede itself ($res(op) < inv(op)$ is impossible).
2. *No 2-cycles:* If $op \longrightarrow op' \longrightarrow op$, it cannot be that both edges are from the same $\longrightarrow_X$ or $\longrightarrow_H$ (as those are acyclic). It also cannot be across different objects X and Y. The only possibility is $op \longrightarrow_X op' \longrightarrow_H op$. But if $op' \longrightarrow_H op$, then $res(op') <_H inv(op)$. In a linearization of X, this would force $op' \longrightarrow_X op$, creating a cycle in the sequential order of X, which is a contradiction.
3. *No larger cycles:* By induction and the properties of the shortest cycle, we can show that any cycle would eventually force a contradiction of the real-time order $\longrightarrow_H$ or the individual object semantics.

Since $\longrightarrow$ is a Directed Acyclic Graph (DAG), it admits a topological order ($\longrightarrow'$). We can then construct our global sequential history $\widehat{S}$ by arranging operations according to this topological order. This constructed history $\widehat{S}$ will naturally satisfy object semantics (matching $\widehat{S}_X$), process consistency (matching p's sequential nature), and real-time order (since $\longrightarrow_H \subseteq \longrightarrow \subseteq \longrightarrow'$).

Example 5.1.2 – Composition of a Queue and a Stack

Consider processes p_1, p_2, p_3 accessing a Queue Q and a Stack S.

- p_1 interacts with Q and S.
- p_2 interacts with Q.
- p_3 interacts with S.

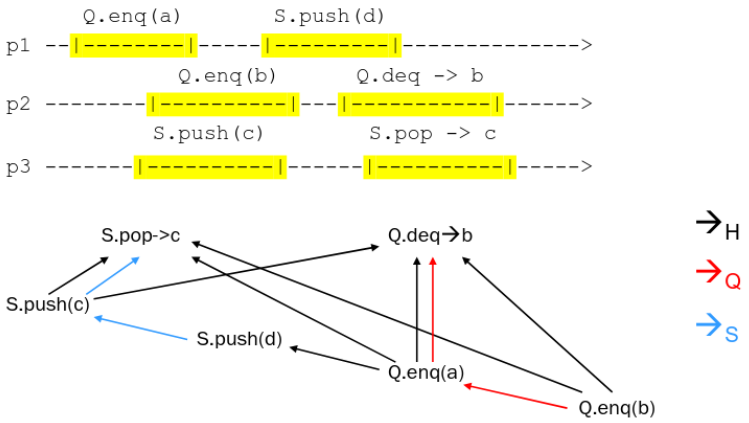


Figure 5.3: Compositionality Timeline and DAG.

By verifying that the operations on Q are linearizable and the operations on S are linearizable independently, compositionality ensures the entire interleaved execution is linearizable. We can find a total order (e.g., `Q.enq(b)`, `Q.enq(a)`, `S.push(d)`, `S.push(c)`, ...) that respects the internal logic of both the stack and the queue while maintaining the real-time precedence of the original execution.

5.2 Alternatives to Atomicity

While linearizability is robust, other consistency models exist, often relaxing the strict real-time requirement to allow for more performance or different system behaviors.

5.2.1 Sequential Consistency

Sequential consistency is a more "generous" or "lazy" version of linearizability. It relaxes the third condition: instead of respecting the global real-time order $\longrightarrow_H$, it only respects the process order $\longrightarrow_{proc}$.

Definition 5.2.1 – Complete sequential history

A complete history $\widehat{H}$ is *sequentially consistent* if there exists a sequential history $\widehat{S}$ such that object semantics and process orders are preserved, but only $op \longrightarrow_{proc} op' \implies op \longrightarrow_S op'$.

1. $\forall X \widehat{S}|_X \in semantics(X)$
2. $\forall p \widehat{H}|_p = \widehat{S}_p$
3. $\longrightarrow_{proc} \subseteq \longrightarrow_S$

Example 5.2.1 – Sequential non-linearizable history

An example of a history that is sequentially consistent but not linearizable is a queue where p_1 enqueues a and p_2 subsequently enqueues b and then dequeues b.

$$[Q.enq(a)() \text{ by } p_1][Q.enq(b)() \text{ by } p_2][Q.deq()(b) \text{ to } p_2]$$

In real-time, a was in the queue first. Linearizability would require a to be dequeued first. Sequential consistency, however, allows us to "reorder" the enqueues in our minds so that b was enqueued before a, thus satisfying the queue semantics.

Sequential consistency is not compositional. Two histories can be individually sequentially consistent, but their composition may not be.

Example 5.2.2 – Non-compositional sequential consistency

$$p_1 : Q.enq(a); Q'.enq(b'); Q'.deq() \to b'$$
$$p_2 : Q'.enq(a'); Q.enq(b); Q.deq() \to b$$

If two processes p_1 and p_2 interact with two different queues Q and Q' in a cross-dependency (e.g., p_1 enqueues to Q then Q', p_2 enqueues to Q' then Q), the local sequential orderings required to satisfy each queue might contradict each other when combined into a single global order.

5.2.2 Serializability

Commonly used in database systems, *serializability* shifts the focus from processes to transactions.

Operations are grouped into transactions, which end with either a `commit(t)` or `abort(t)` event.

Definition 5.2.2 – Serializability

A history is *serializable* if there is a sequential history $\widehat{S}$ (consisting of only committed transactions) that preserves object semantics and the transaction order $\longrightarrow_{trans}$ (where one transaction finishes before another begins).

1. $\forall X \widehat{S}|_X \in semantics(X)$
2. $S = \{e \in H : e \in t \in committedTrans(\widehat{H})\}$
3. $\longrightarrow_{trans} \subseteq \longrightarrow_S$

Like sequential consistency, serializability is more flexible than linearizability because it focuses on the logical order of transactions rather than the wall-clock time of every individual event. However, it shares the same critical flaw: it is not compositional. Combining two serializable database systems does not automatically result in a serializable global system without additional coordination.

Chapter 6

MUTEX-free Concurrency

Traditional concurrent systems rely heavily on critical sections and locks to ensure safety. However, this approach introduces several systemic risks. If locks are implemented with improper granularity, they can lead to performance bottlenecks by unnecessarily reducing concurrency. More critically, the delay or failure of a single process—such as a process crashing while holding a lock—can stall the entire system indefinitely.

Mutex-freedom addresses these issues by ensuring that the only atomic operations used are those provided directly by the hardware (such as atomic Read/Write registers, Compare-and-Swap, or Fetch-and-Add). Because these systems do not use traditional critical sections, we require a new set of liveness properties to define progress. These properties are particularly robust in the face of process crashes (fail-stop failures), as asynchrony makes it impossible for the system to distinguish between a crashed process and one that is simply delayed.

The four primary progress conditions are:

1. *Obstruction-freedom:* This is the weakest condition. It guarantees that an operation will terminate if it eventually executes in isolation (i.e., without contention from other processes for a sufficient period).
2. *Non-blocking (Lock-free):* This ensures that at least one of the active operations on an object will eventually terminate. While individual processes might still face starvation, the system as a whole always makes progress. This is analogous to deadlock-freedom in lock-based systems.
3. *Wait-freedom:* This is the strongest condition, ensuring that every operation invoked by a process will eventually terminate, regardless of the speed or interference of other processes. This is the mutex-free equivalent of starvation-freedom.
4. *Bounded Wait-freedom:* This is a stronger version of wait-freedom where there is a known upper bound on the number of steps a process must take before finishing. This mirrors the concept of bounded bypass.

6.1 Wait-free Splitter

A *splitter* is a fundamental building block in distributed computing used to "partition" processes. It provides a single operation, `dir(i)`, which returns one of three values: L (Left), R (Right), or S (Stop).

For a splitter to be valid, it must satisfy three properties:

- *Validity:* It must return L, R, or S.
- *Concurrency:* If n processes invoke the splitter, at most $n-1$ can go `Left`, at most $n-1$ can go `Right`, and at most one process can obtain the `Stop` value.
- *Wait-freedom:* Every invocation must terminate.

In a "solo" execution where only one process enters the splitter, that process is guaranteed to receive the S value. This makes the splitter highly effective for resource allocation or leadership election in a distributed environment.

The implementation uses two atomic registers: DOOR (a boolean initialized to 1) and LAST (initialized to a default process index).

```
DOOR: MRMW boolean atomic register initialized at 1
LAST: MRMW atomic register initialized at whatever process index
```

```
dir(i) :=
    LAST ← i
    if DOOR = 0 then return R
                else DOOR ← 0
                    if LAST = i then return S
                                else return L
```

Listing 6.1: Wait-free Splitter.

The logic follows a specific sequence: a process first announces its arrival by writing its ID to LAST. It then checks the DOOR. If the door is already closed (DOOR = 0), it immediately exits to the Right (R). If the door is open, the process closes it and then checks if it is still the "last" process to have written to LAST. If its ID remains in LAST, it stops (S); otherwise, it concludes that another process has overtaken it and moves to the Left (L).

Theorem 6.1.1 – Soundness proof for wait-free Splitter

The termination and validity of the splitter are straightforward due to the lack of loops.

Demonstration. The concurrency property is proved through three observations:

- Not everyone can go Right: To receive R, a process must find the DOOR at 0. The first process to find the door at 1 will close it and cannot receive R.
- Not everyone can go Left: Consider the last process p_i to write to the LAST register. If p_i finds the door closed, it goes R. If it finds the door open, it will inevitably find LAST = i (since no one writes after it) and will receive S.
- At most one process stops: Suppose p_i is the first process to receive S. This means p_i found LAST = i during its second check. Any process p_j that wrote to LAST before p_i will find LAST has changed (to i or a later value) and will go L. Any process p_j that tries to write to LAST after p_i has already closed the door will find DOOR = 0 and go R.

6.2 Obstruction-free Timestamp Generator

A *timestamp generator* provides a get_ts operation that must ensure:

- *Validity:* two invocations of get_ts does not return the same value.
- *Consistency:* if one process finishes getting a timestamp before another begins, the first must receive a smaller value.
- *Obstrucion freedom:* if run in isolation, it eventually terminates.

The proposed implementation uses an array of "splitters" (represented by DOOR[k] and LAST[k]) and a shared integer NEXT to track the current timestamp being contested.

```
DOOR[i]: MRMW boolean atomic register initialized at 1, for all i
LAST[i]: MRMW atomic register initialized at whatever process index, for all i
NEXT: integer initialized at 1
```

```
get_ts(i) :=
    k ← NEXT
    while true do
        LAST[k] ← i
        if DOOR[k] = 1 then
            DOOR[k] ← 0
            if LAST[k] = i then NEXT++
                                return k
        k++
```

Listing 6.2: Obstruction-free Timestamp Generator.

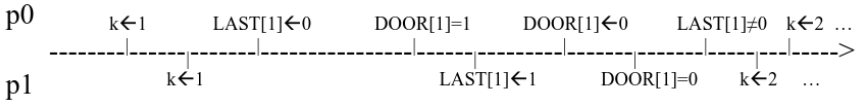
In this algorithm, a process p_i starts at the current value of NEXT. It attempts to "win" that timestamp by acting as it would in a splitter. If it fails to get the "Stop" equivalent (i.e., it doesn't return k), it increments k and tries the next available slot. When a process successfully secures a timestamp k, it increments NEXT to $k+1$ to guide future processes toward higher values.

Theorem 6.2.1 – Obstruction-free Timestamp Generator

Demonstration. The generator is

- Valid because only one process can "stop" at any index k (based on the splitter property).
- Consistent because a process only returns k after setting NEXT to $k+1$. Any process starting later will read this updated NEXT and thus receive a timestamp $\geq k+1$.

However, this implementation is only obstruction-free, not non-blocking. Under specific patterns of interference, processes can continually displace each other.



For example, if p_0 and p_1 both enter the loop for $k=1$, p_0 might write to LAST[1] and close the door DOOR←0. Then p_1 writes to LAST[1], causing p_0 to fail the LAST[k] = i check and p_1 finds the door closed. If they both then move to $k=2$ and repeat this exact timing, they could theoretically loop forever. This lack of progress only occurs during active contention; if one process were to run in isolation, it would successfully secure the next available k and terminate.

6.3 Wait-free Stack

A wait-free implementation of a stack can be constructed using an unbounded array of atomic registers, denoted as REG. In this model, every register in the array is initialized to $\perp$ and supports atomic writes as well as a swap(v) primitive. The swap operation is crucial as it atomically writes a new value into a register and returns the value that was previously stored there. To manage the current position of the stack, a shared atomic register called NEXT is used, which is initialized to 1 and supports fetch&add operations.

The operations are defined as follows:

- push(v): A process first calls NEXT.fetch&add(1) to reserve a unique index i in the REG array. It then writes the value v into REG[i]. Because fetch&add is atomic, every push operation is guaranteed a unique slot, ensuring that no two processes overwrite each other's data before it can be processed.

```
1 push(v) :=
2     i ← NEXT.fetch&add(1)
3     REG[i] ← v
```

Listing 6.3: Wait-free Stack `push` operation.

- `pop()`: A process begins by reading the current value of `NEXT` and setting `k=NEXT-1`. It then iterates downwards from k to 1, attempting to "claim" a value by performing `REG[i].swap(`$\perp$`)`. If the swap returns a value other than $\perp$, that value is returned as the popped element. If the loop completes without finding a value, it returns $\perp$, indicating the stack is effectively empty.

```
1 pop() :=
2     k ← NEXT-1
3     for i=k downto 1
4         tmp ← REG[i].swap(⊥)
5         if tmp≠ ⊥ then return tmp
6     return ⊥
```

Listing 6.4: Wait-free Stack `pop` operation.

A significant advantage of this implementation is that process crashes do not compromise the progress of the system. Since the operations are wait-free, every invocation eventually terminates regardless of the behavior of other processes. However, this specific design relies on the unboundedness of the REG array, which is not a realistic assumption for physical hardware.

6.4 Non-blocking Bounded Stack

To address the limitations of unbounded memory, we can implement a *bounded stack* that satisfies the non-blocking progress condition. The core idea behind this implementation is "helping": every operation started by an invoking process is finalized either by that process or by the subsequent process that enters the system.

This implementation utilizes an array `STACK[0..k]` and a `TOP` register, both of which support compare&set (CAS) operations. However, using CAS introduces the *ABA problem*. This occurs when a process reads a value A, and before it can perform a CAS to change A to C, other processes change the value from A to B and then back to A. The CAS would succeed, potentially ignoring the intermediate state change.

To solve this, we use versioning or sequence numbers. In this bounded stack:

- Each entry in `STACK[i]` is a pair $\langle val, seq\_numb \rangle$, initialized to $\langle \perp, 0 \rangle$. Any modification to a stack slot must increment its sequence number.

  The `TOP` register is a triple $\langle index, val, seq\_numb \rangle$, initialized to $\langle 0, \perp, 1 \rangle$. Here, *index* represents the current top of the stack, *val* is the value to be placed there, and *seq_numb* ensures that every update to the top of the stack is unique.

The implementation relies on three primary functions:

- `conclude(i, v, s)` is designed to ensure that the value associated with a stack index is properly written into the `STACK` array. It reads the current value at `STACK[i]` and attempts to compare&set it to the intended value v while incrementing the sequence number to s. This ensures that even if a process crashes immediately after updating `TOP`, the next process to access the stack will finish the work.

```
1 conclude(i,v,s) :=
2     tmp ← STACK[i].val
3     STACK[i].compare&set(⟨tmp,s-1⟩,⟨v,s⟩)
```

Listing 6.5: Non-blocking Bounded Stack `conclude` operation.

- `push(w)`: The process enters a loop where it reads the `TOP` triple. It first calls conclude to ensure any previous operation is finished. If the index i has reached the limit k, it returns `FULL`. Otherwise, it calculates a `newtop` containing the incremented index and the new value w. It then attempts a `TOP.compare&set`. If it succeeds, the push is logically complete.

```
1 push(w) :=
2     while true do
3         ⟨i,v,s⟩ ← TOP
4         conclude(i,v,s)
5         if i=k then return FULL
6         newtop ← ⟨i+1,w,STACK[i+1].seq_num+1⟩
7         if TOP.compare&set(⟨i,v,s⟩,newtop)
8         then return OK
```

Listing 6.6: Non-blocking Bounded Stack `push` operation.

- `pop()`: Similar to push, the process reads `TOP` and calls conclude. If the index is 0, it returns `EMPTY`. It then attempts to move `TOP` to the previous index $(i - 1)$ using compare&set. If successful, it returns the value v that was at the top.

```
1 pop() :=
2     while true do
3         ⟨i,v,s⟩ ← TOP
4         conclude(i,v,s)
5         if i=0 then return EMPTY
6         newtop ← ⟨i-1,STACK[i-1]⟩
7         if TOP.compare&set(⟨i,v,s⟩,newtop)
8         then return v
```

Listing 6.7: Non-blocking Bounded Stack `pop` operation.

Theorem 6.4.1 – Liveness proof in Non-blocking Bounded Stack

The liveness of this implementation is guaranteed to be non-blocking.

Demonstration. If a process p fails to complete its compare&set on `TOP`, it is because the value of `TOP` has changed in the interim. Since the only instruction that modifies `TOP` is the final compare&set of a successful push or pop, a failure by p implies that at least one other operation has successfully terminated. This satisfies the non-blocking requirement that the system as a whole makes progress.
The use of the `conclude` function is vital for correctness during crash failures. In a naive implementation, if a process crashed after updating `TOP` but before writing the value into the actual stack array, the stack would be left in an inconsistent state. By making the "conclusion" of an operation a shared responsibility—performed both at the end of the current invocation and at the start of the next—the system ensures that the data is always consistently applied before the stack is modified again.

6.5 Enhancing Liveness Properties

In mutual exclusion (MUTEX)-based concurrency, it is a known principle that weak liveness properties, such as deadlock-freedom, can be elevated to stronger ones like bounded bypass. The goal is to apply a similar logic to mutex-free concurrency. Mutex-freedom is defined by the absence of critical sections or locks in the implementation of a concurrent object, ensuring that no process is blocked because another process has paused or crashed while holding a resource.

To achieve stronger liveness in this framework, we introduce the concept of a *Contention Manager*. This is a specialized object designed to provide "contention-free periods" or "favorable circumstances" that allow a process to complete its operation without being repeatedly interrupted by others. A contention manager typically provides two primary operations:

- `need_help(i)`: Invoked by process p_i when it detects contention (e.g., its operation has failed to progress after multiple attempts).
- `stop_help(i)`: Invoked by p_i once it has successfully completed its operation.

This enrichment differs from traditional lock/unlock mechanisms because it must account for fail-stop failures. In a mutex-free system, a process can crash during a contention-free period. Because an asynchronous system cannot inherently distinguish between a crashed process and one that is simply experiencing a long delay, we must rely on *failure detectors*. These modules provide processes with (potentially unreliable) information about which other processes in the system have failed, and they are categorized by the quality and accuracy of the information they provide.

6.5.1 From obstruction-freedom to non-blocking

Obstruction-freedom is a weak progress condition where a process is guaranteed to finish its operation only if it eventually executes in isolation for a sufficient amount of time. To "boost" this to a non-blocking property—where at least one process is always guaranteed to make progress—we use a failure detector called *Eventually Restricted Leadership* (Ω_X).

Given a set of process IDs X, the Ω_X detector provides each process with a local variable, `ev_leader(X)`, which satisfies two properties:

- *Validity:* The variable always contains a valid process ID from the set.
- *Eventual Leadership:* Eventually, all non-crashed processes in X will permanently agree on the same leader, who must itself be a non-crashed process.

The implementation of this contention manager uses an array of Single-Writer Multiple-Reader (SWMR) atomic registers, `NEED_HELP[1..n]`, initialized to `false`.

```
1 NEED_HELP[1..n] : SWMR atomic R/W boolean registers init at false
```

```
1 need_help(i) :=
2     NEED_HELP[i] ← true
3     repeat
4         X ← {j : NEED_HELP[j]}
5     until ev_leader(X) = i
```

```
1 stop_help(i) :=
2     NEED_HELP[i] ← false
```

Listing 6.8: From obstruction-freedom to non-blocking.

When a process p_i calls `need_help(i)`, it sets its register to true and then enters a loop. Inside this loop, it identifies the set X of all processes currently requesting help and waits until the failure detector Ω_X designates p_i as the leader. Once p_i finishes its task, it calls `stop_help(i)` to set its register back to `false`.

Theorem 6.5.1 – From obstruction-freedom to non-blocking

The contention manager described above successfully transforms an obstruction-free implementation into a non-blocking one.

Demonstration. This is proven by contradiction: assume there is a time τ after which several concurrent operations never terminate. Let Q be the set of processes attempting these operations.

- Due to the enrichment, eventually all processes in Q will set their `NEED_HELP` flags to `true` forever.
- Crashed processes outside of Q will eventually stop modifying their flags.
- This leads to a point where all processes in Q compute the same set X of processes needing help.
- By the definition of Ω_X, a single leader will eventually be elected from X (and since the leader must be non-crashed, it must be in Q).

- This leader will then be the only process allowed to proceed, effectively running in isolation. Because the underlying implementation is obstruction-free, the leader is guaranteed to terminate, contradicting the assumption that no process in Q makes progress.

Implementing Ω is impossible in a purely asynchronous system because crashes are indistinguishable from long delays. Therefore, we must introduce timing constraints. One common approach assumes:

1. *Bounded Write Delay:* There exists a correct process p_L and a time interval Δ such that p_L writes to its progress register at least once every Δ time units.

2. *Timer Accuracy:* There are enough correct processes with timers that will eventually not expire "too early," allowing them to correctly identify the progress of p_L.

The implementation idea involves a `PROGRESS[1..n]` array where each process regularly increments its entry to signal it is alive. If process p_i does not see progress from p_j within a certain (guessed) time interval, it "suspects" p_j. The leader is then chosen as the "least suspected" process. To handle the uncertainty of the system's speed, processes dynamically adjust their suspicion timers by tracking the number of times they have suspected others (`SUSPECT[i, j]`) and summing these values to guess a duration that eventually exceeds the actual system delay Δ.

6.5.2 From obstruction-freedom to wait-freedom

To achieve wait-freedom—where every correct process is guaranteed to finish its operation in a finite number of steps—a stronger failure detector is required: the *Eventually Perfect Failure Detector* ($\Diamond P$). $\Diamond P$ provides:

- *Eventual Completeness:* Eventually, every crashed process is suspected by every correct process.

- *Eventual Accuracy:* Eventually, no correct process is suspected by any correct process.

In the hierarchy of failure detectors, $\Diamond P$ is strictly stronger than Ω_X. We can build Ω_X from $\Diamond P$ by having each process pick the process with the minimum ID among those not currently suspected. However, Ω_X cannot build $\Diamond P$ because Ω_X only guarantees one common leader, while $\Diamond P$ requires eventual perfect knowledge of all crashes.

The wait-free manager utilizes a weak timestamp generator, which ensures that for any timestamp t returned, only a finite number of subsequent requests can receive a timestamp smaller than or equal to t. The algorithm works as follows:

```
1 TS[1..n] : SWMR atomic R/W registers init at 0

1 need_help(i) :=
2     TS[i] ← weak_ts()
3     repeat
4         competing ← {j : TS[j]≠0 ∧ j ∉ suspected_i}
5         ⟨t,j⟩ ← min{⟨TS[x],x⟩ | x ∈ competing}
6     until j = i

1 stop_help(i) :=
2     TS[i] ← 0
```

Listing 6.9: From obstruction-freedom to wait-freedom.

- `need_help(i)`: Process p_i obtains a timestamp `TS[i]`. It then loops, identifying a set of "competing" processes (those with non-zero timestamps who are not currently suspected by the failure detector). It waits until its own timestamp-identity pair $\langle TS[i], i\rangle$ is the minimum among all competitors.

- `stop_help(i)`: Once the operation is done, p_i resets `TS[i]` to 0.

Theorem 6.5.2 – Wait-freedom proof of from obstruction-freedom to wait-freedom

Demonstration. Suppose a correct process p_i never terminates its invocation. Let its timestamp be t_i, and assume $\langle t_i, i \rangle$ is the minimum such pair that fails to terminate.

- Any process that crashes will eventually be suspected by p_i (due to $\diamondsuit P$ completeness) and will be removed from the competing set.
- Any process with a smaller timestamp than p_i must either crash or terminate (since p_i is the minimum non-terminating case).
- Once these finite "older" invocations are cleared and crashed processes are suspected, p_i will eventually have the minimum timestamp and be allowed to run in isolation.
- Because the implementation is obstruction-free, p_i will eventually finish.

To implement $\diamondsuit P$ itself, the system relies on processes having an eventual upper bound on write delays. By adjusting timers based on whether a process's suspicion was correct or "premature," the system eventually reaches a state where only actually crashed processes are suspected.

Chapter 7

Universal Object

In the study of concurrent systems, a fundamental question arises regarding the implementation of complex objects: given objects of a specific type T and a basic object type Z, is it possible to create a wait-free implementation of Z using only objects of type T and atomic Read/Write (R/W) registers? This question addresses the relative "synchronization power" of different object types.

To answer this, we must first define what constitutes a "type." An object type is defined by two primary components:

1. *The State Space:* The set of all possible values that an object of that type can hold.
2. *The Operations:* A set of operations used to manipulate the object, each accompanied by a sequential specification. This specification describes the conditions for invocation and the resulting effect on the object's state.

In this context, we focus on types with operations that are *total* and *sequentially specified*. An operation is "total" if it can be invoked regardless of the current state of the object. A sequential specification implies that the object's behavior is entirely determined by the sequence of operations performed on it. Formally, this is represented by a transition function $\delta(s, op(args)) = \{\langle s_1, res_1\rangle, \ldots, \langle s_k, res_k\rangle\}$. In a *deterministic object*, $k = 1$, meaning for every state and operation, there is exactly one resulting state and one output.

An object type T_U is considered *universal* if it is powerful enough to implement any other object type in a wait-free manner, using only T_U and atomic R/W registers.

7.1 Consensus Object

A *consensus object* is a "one-shot" object—meaning each process can access it only once. It provides a single operation, `propose(v)`, which must satisfy four critical properties:

- *Validity:* The value decided upon must have been proposed by at least one participant.
- *Integrity:* Each process can decide on a value at most once.
- *Agreement:* All processes that decide must decide on the same value.
- *Wait-freedom:* Any correct process that invokes the operation will eventually terminate.

```
1 propose(v) := if X = ⊥ then X ← v
2               return X
```

Listing 7.1: Consensus object implementation.

Conceptually, one could think of implementing a consensus object using a register X initialized to a null value ($\perp$). The `propose(v)` operation would atomically check if X is $\perp$; if so, it writes v to X. Regardless of whether the write occurred, the operation returns the value currently in X. This "winner-takes-all" logic ensures that the first value written becomes the permanent decision.

The core thesis of the universality of consensus is that if we have consensus objects, we can implement any other object O of type Z. This is achieved by having every participant run a local copy of O,

all starting at the same initial state. By using consensus objects to create a total order of operations, processes can "force" their local simulations to follow the exact same sequence. Because the objects are deterministic, consistent local copies result in a consistent global state.

To understand the wait-free implementation, we first look at a simpler, non-blocking (but not wait-free) construction for deterministic objects.

```
1 result_i ← ⊥
2 invoc_i ← ⟨op(args), i⟩
3 wait result_i ≠ ⊥
4 return result_i
```

Listing 7.2: `op(arg)` by p_i on Z for non-blocking, not wait-free deterministic objects.

In this model, a process p_i wanting to execute an operation `op(args)` on an object Z uses two local variables: $\texttt{result}_i$ (to store the final output) and $\texttt{invoc}_i$ (to store the operation and the process ID).

The system uses an unbounded array of consensus objects, `CONS`.

```
1  k ← 0
2  z_i ← Z.init()
3  while true
4      if invoc_i ≠ ⊥ then
5          k++
6          exec_i ← CONS[k].propose(invoc_i)
7          ⟨z_i, res⟩ ← δ(z_i, π_1(exec_i))
8          if π_2(exec_i) = i then
9              invoc_i ← ⊥
10             result_i ← res
```

Listing 7.3: Local simulation of Z by p_i for non-blocking, non wait-free deterministic objects.

Each process runs a simulation of Z in a local variable z_i. The logic proceeds in a loop:

- Process p_i increments a counter k to move to the next consensus instance.
- It proposes its operation $\texttt{invoc}_i$ to `CONS[k]`.
- The consensus object returns a winning operation, $\texttt{exec}_i$. This may be p_i's operation or an operation proposed by another process.
- Process p_i applies $\texttt{exec}_i$ to its local state z_i using the transition function δ.
- If the operation that won the consensus was indeed p_i's operation ($\pi_2(exec_i) = i$), p_i updates its $\texttt{result}_i$ and clears its pending invocation.

While this ensures that all processes see the same operations in the same order, it is not wait-free. A process could potentially be "starved" if it consistently loses the consensus to other processes, preventing its operation from ever being executed.

To achieve wait-freedom, the construction must ensure that every process's operation is eventually executed, even if the process itself does not "win" the consensus. This is handled through a "helping" mechanism.

The shared state involves an array of Single-Writer/Multi-Reader (SWMR) atomic registers called `LAST_OP[1..n]`. Each register i stores a pair: the operation p_i wants to perform and a sequence number. Locally, each process p_j maintains an array `last_sn_j[1..n]` to track which operations from other processes it has already executed.

```
1 result_i ← ⊥
2 LAST_OP[i] ← ⟨op(args),
3                last_sn_i[i]+1⟩
4 wait result_i ≠ ⊥
5 return result_i
```

Listing 7.4: `op(arg)` by p_i on Z for non-blocking, wait-free deterministic objects.

```
1  k ← 0
2  z_i ← Z.init()
3  while true
4      invoc_i ← ϵ
5      ∀ j = 1..n
6          if π_2(LAST_OP[j])>last_sn_i[j]
7          then invoc_i.append(
8                  ⟨π_1(LAST_OP[j]),j⟩ )
9      if invoc_i ≠ ϵ then
10         k++
11         exec_i ← CONS[k].propose(invoc_i)
12         for r=1 to |exec_i|
13             ⟨z_i,res⟩ ← δ(z_i,π_1(exec_i[r]))
14             j ← π_2(exec_i[r])
15             last_sn_i[j]++
16             if i=j then result_i ← res
```

Listing 7.5: Local simulation of Z by p_i for non-blocking, wait-free deterministic objects.

In this construction, the simulation loop is more complex. When a process p_i wants to perform an operation, it first announces it by writing to `LAST_OP[i]` with an incremented sequence number. It then enters the simulation loop. Instead of proposing a single operation to the consensus object, it proposes a list of all pending operations it sees in the `LAST_OP` array (those where the sequence number in the register is greater than the one it has locally recorded).

The consensus object `CONS[k]` now decides on a list of operations. When p_i receives the winning list $\texttt{exec}_i$, it iterates through the entire list, applying each operation to its local state z_i and updating the corresponding sequence numbers in $\texttt{last\_sn\_i}$. If p_i finds its own operation within the list, it can finally return the result.

The correctness of this construction is established through three primary lemmas:

Lemma 7.1.1 – The construction is wait-free

The construction is wait-free.

Demonstration. To prove wait-freedom, we must show that p_i eventually finds its operation in a decided list. Because p_i publishes its operation in `LAST_OP[i]` with an increased sequence number, any process (including p_i itself) that participates in a subsequent consensus will see this operation and include it in their proposed list. Since at least one correct process will eventually succeed in a consensus round, p_i's operation is guaranteed to be part of an agreed-upon list.

Lemma 7.1.2 – All operations are executed exactly once

All operations are executed exactly once.

Demonstration. We must ensure that an operation isn't applied to the state multiple times. p_i cannot start a new operation until its current one is executed. Furthermore, every process tracks sequence numbers locally (`last_sn`). When a process processes a list from consensus, it only appends operations to its "to-be-executed" list if the sequence number is new. If a process p_j participates in consensus k, it updates its local sequence numbers before forming a proposal for consensus $k+1$, ensuring it doesn't propose the same operation twice.

Lemma 7.1.3 – Consistency and Sequential Specification

Consistency and sequential specification.

Demonstration. Because all processes access the consensus objects in the same order (`CONS[1]`, `CONS[2]`, ...) and consensus guarantees all processes receive the same list, every process applies the same sequence of operations to their local copies. Since the starting states were identical and the transitions are deterministic, the local states z_i remain consistent across

all processes.

It is important to note that while this construction is wait-free, it does not provide bounded wait-freedom. If a process p_i is suspended for a significant amount of time, it may wake up to find an immense "log" of agreed-upon lists in the `CONS` array. It must catch up by locally executing all these operations before it can complete its own current operation. This catch-up phase can be arbitrarily long, meaning the time to complete an operation is not strictly bounded.

The previous wait-free constructions assume that the object Z is deterministic, meaning that for any given state and operation, there is exactly one possible resulting state and output. However, many concurrent objects have non-deterministic specifications where the transition function $\delta(s, op(args))$ returns a set of possible pairs $\{\langle s_1, res_1\rangle, \ldots \langle s_k, res_k\rangle\}$. In such cases, if processes simply agree on the order of operations, their local simulations might diverge if they independently pick different outcomes from the set of possibilities.

To maintain consistency across all local copies in a wait-free manner, processes must be forced to choose the same transition for every non-deterministic step. The slides suggest three potential solutions:

1. *The "Brute Force" Solution:* This approach involves "canceling" the non-determinism by fixing a rule a priori. For every possible pair of state and operation, one specific outcome from the set δ is pre-selected as the "correct" one. This effectively turns a non-deterministic object into a deterministic one before the simulation begins.

2. *Additional Consensus Objects:* A more dynamic approach is to use a new consensus object for every single operation. These additional consensus instances would be used specifically to agree on which outcome ($\langle s_i, res_i\rangle$) to choose from the set of valid transitions returned by δ.

3. *Reusing the Primary Consensus Object:* Instead of using separate consensus objects for the choice of outcome, the proposals sent to the main consensus array (`CONS[k]`) can be expanded. A process would not only propose the list of operations to be performed but also pre-calculate and propose the resulting state and return value for each operation in that list. This ensures that when the consensus returns an agreed-upon list, it also includes the agreed-upon results, maintaining local consistency.

7.2 From Binary to Multivalued Consensus

A fundamental question in synchronization is whether the complexity of the data we agree upon matters. *Binary consensus* allows only two possible proposals (typically 0 and 1). *Multivalued consensus* allows for an unbounded set of possible values. We can implement a multivalued consensus object using only binary consensus objects and registers.

The construction for *unbounded multivalued consensus* (where the number of possible values is not fixed) uses an array of n binary consensus objects (`BC[1..n]`) and an array of n proposal registers (`PROP[1..n]`).

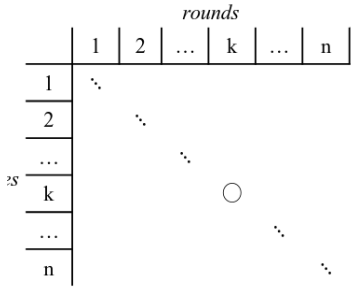


Figure 7.1: Multivalued consensus (with unbounded values).

```
1 PROP[1..n] array of n proposals, all init at ⊥
2 BC[1..n] array of n binary consensus objects

1 mv_propose(i,v) :=
2     PROP[i] ← v
3     for k=1 to n do
4         if PROP[k] ≠ ⊥ then res ← BC[k].propose(1)
5                        else res ← BC[k].propose(0)
6         if res = 1 then return PROP[k]
7     wait forever
```

Listing 7.6: Multivalued consensus (with unbounded values).

Each process p_i proposing a value v follows this logic:

1. It writes its value v to its specific register `PROP[i]`.
2. It then iterates through all processes from $k = 1$ to n.
3. For each k, it checks if process p_k has written a proposal. If `PROP[k]` is not empty, it proposes 1 to the binary consensus `BC[k]`. Otherwise, it proposes 0.
4. If the binary consensus `BC[k]` returns 1, it means the system has agreed to use the value proposed by p_k. Process p_i then returns `PROP[k]` as the decided value.

Demonstration 7.2.1 – Proof of Correcteness for Multivalued Consensus (with unbounded values)

- *Validity:* If a process p_x is the first to write a proposal, any process entering the loop later will see `PROP[x]` is not empty and propose 1 to `BC[x]`. Thus, at least one binary consensus will eventually return 1.
- *Agreement:* Because every process iterates through the binary consensus objects in the same order (1 to n), they will all stop at the same first index y where `BC[y]` returns 1. Consequently, they all decide on the same value `PROP[y]`.

If the number of possible values k is known and bounded, we can use a more efficient construction. By representing each value as a binary string of length $h = \log_2 k$, we can decide the final value one bit at a time.

In this *bounded construction*, we use an array of h binary consensus objects (`BC[1..h]`). Each process p_i writes its proposal as a bit-string in `PROP[i]`.

```
1 PROP[1..n][1..h] array of n h-bits proposals, all init at ⊥
2 BC[1..h] array of h binary consensus objects

1 bmv_propose(i,v) :=
2     PROP[i] ← binary_repr_h(v)
3     for k=1 to h do
4         P ← {PROP[j][k] | PROP[j] ≠ ⊥ ∧ PROP[j][1..k-1]=res[1..k-1]}
5         let b be an element of P
6         res[k] ← BC[k].propose(b)
7     return value(res)
```

Listing 7.7: Multivalued consensus (with bounded values).

The processes then loop from $k = 1$ to h to decide each bit:

1. In each round k, a process identifies the set P of possible bits. This set contains the k-th bits of all proposals currently in the registers that match the prefix already decided in rounds 1 to $k - 1$.
2. The process picks a bit b from this set P and proposes it to `BC[k]`.
3. The result of `BC[k]` becomes the k-th bit of the final decided value.

Demonstration 7.2.2 – Proof of Correctness Analysis for Multivalued consensus (with bounded values)

- *Wait-freedom and Integrity:* These are inherited from the underlying binary consensus objects and the fixed number of iterations (h).
- *Agreement:* This is guaranteed because each bit is decided by a binary consensus object, which ensures all processes see the same bit for each position in the string.
- *Validity:* The construction of the set P is vital. By only proposing bits that belong to a proposal matching the existing prefix, the algorithm ensures that the final "assembled" bit-string is not a hybrid of different proposals, but rather matches at least one full proposal that was actually submitted by a participant. For every bit $b \in P$ selected, there is at least one process p_j whose proposal matches the decided bits up to that point. By the time the loop reaches h, the final string must equal some p_j's original proposal.

Chapter 8

Consensus Number

In the study of concurrent systems, a fundamental question arises: which types of shared objects allow for a wait-free implementation of binary consensus? The answer is not absolute; rather, it depends entirely on the number of participating processes. To categorize the "synchronization power" of different object types, we use the concept of a consensus number.

The *consensus number* of an object of type T, denoted as $CN(T)$, is defined as the greatest number n such that it is possible to provide a wait-free implementation of a consensus object in a system of n processes using only objects of type T and standard atomic read/write (R/W) registers. For every object type T, the consensus number is at least 1, as a single process can always "decide" its own proposal. If there is no upper bound on the number of processes for which a wait-free implementation exists, we say $CN(T) = \infty$.

This definition leads to a powerful hierarchy theorem.

Theorem 8.0.1 – Consensus Number

If $CN(T_1) < CN(T_2)$, then there is no wait-free implementation of objects of type T_2 in a system of n processes that uses only objects of type T_1 and atomic R/W registers, provided that $CN(T_1) < n \leq CN(T_2)$. This theorem establishes that object types with higher consensus numbers are strictly more powerful than those with lower ones.

Demonstration. The proof of this hierarchy relies on contradiction. If we assume such an implementation exists, then because $n \leq CN(T_2)$, we know consensus can be implemented using T_2. By substituting the T_1-based implementation for T_2, we would effectively have a wait-free consensus for n processes using only T_1. However, this contradicts the definition of $CN(T_1)$, which states that T_1 cannot solve consensus for any number of processes greater than $CN(T_1)$.

8.1 Schedules and Configurations

To formally prove the limits of consensus, we must define the state of the system. A *Schedule* (S) is a sequence of operation invocations issued by processes. A *Configuration* (C) represents the global state of the system at any given time, encompassing the values of all shared memory objects and the local state of every process. If we apply a schedule S starting from configuration C, the resulting state is denoted as $S(C)$.

When analyzing a binary consensus algorithm A, we categorize configurations based on their potential outcomes:

- *v-valent:* A configuration is v-valent (where $v \in \{0, 1\}$) if every possible execution starting from that configuration eventually decides the value v.
- *Monovalent:* A configuration that is either 0-valent or 1-valent.
- *Bivalent:* A configuration that is not monovalent, meaning that both a decision of 0 and a decision of 1 are still possible depending on the subsequent scheduling of processes.

Theorem 8.1.1 – Fundamental theorem

A critical result in distributed computing is that if an algorithm A wait-free implements binary consensus for n processes, there must exist an initial configuration that is bivalent.

Demonstration. We can prove this by considering a sequence of initial configurations C_0,C_1,...,C_n, where C_i is the configuration where processes p_1 through p_i propose 1, and the remaining processes propose 0. By the validity requirement of consensus, C_0 must be 0-valent (all propose 0) and C_n must be 1-valent (all propose 1).
If we assume for contradiction that all initial configurations are monovalent, there must be some index i where C_{i-1} is 0-valent and C_i is 1-valent. These two configurations differ only in the proposal of process p_i. Now, consider a schedule S where p_i is "asleep" or blocked for a long period. By the wait-freedom property, the other processes must eventually decide even without p_i. In C_{i-1}, they must decide 0. Because p_i is sleeping, the other processes cannot distinguish between C_{i-1} and C_i, so they must also decide 0 in C_i. This leads to a contradiction: if p_i wakes up in C_i and decides 1 (to satisfy its own valence), it violates agreement; if it decides 0, it contradicts the 1-valence of C_i. Thus, an initial bivalent state must exist.

Theorem 8.1.2 – CN(Atomic R/W registers) = 1

Atomic read/write registers are the most basic shared memory objects, and it can be proven that they cannot solve consensus for even two processes ($n = 2$).

Demonstration. To prove *CN(Atomic R/W)=1* by contradiction, we assume an algorithm A exists for two processes, p and q. We start from a bivalent initial configuration C and follow a schedule S until we reach a "maximally bivalent" configuration C'. In C', any move by p leads to a 0-valent state, and any move by q leads to a 1-valent state. We then analyze the possible operations p and q could perform on registers R_1 and R_2:

1. $R_1 \neq R_2$: If they access different registers, their operations commute. The state $q(p(C'))$ would be the same as $p(q(C'))$. However, $q(p(C'))$ must be 0-valent (since $p(C')$ was 0-valent) and $p(q(C'))$ must be 1-valent (since $q(C')$ was 1-valent). This is a contradiction.

2. $R_1 = R_2$ and both read: Reading does not change the shared state. Thus, the global state after both read is the same regardless of order, leading to the same contradiction as case 1.

3. $R_1 = R_2$, one reads and one writes: Suppose p reads and q writes. Only p can see the result of its read (it's a local state change). If we let q run until it decides while p is stopped, q will decide 1 because $q(C')$ is 1-valent. If we then look at the configuration $p(C')$ and let q run the same schedule, q cannot distinguish $p(C')$ from C' because the shared register looks the same. q will decide 1 again. If we then reactivate p, no matter what it decides, it either violates agreement or its own 0-valence.

4. $R_1 = R_2$ and both write: If q writes after p, the value written by p is overwritten and lost. Therefore, $q(p(C'))$ is indistinguishable from $q(C')$ for process q. This leads to the same contradiction as case 3.

Because all cases lead to a contradiction, atomic R/W registers cannot solve consensus for 2 processes.

The test&set (T&S) operation is more powerful. A T&S object maintains a value (initially 0). The operation `test&set()` atomically sets the value to 1 and returns the old value. We can implement 2-process consensus using T&S as follows: Each process $i \in \{0, 1\}$ writes its proposal to a shared array `PROP[i]`. Then, it calls `TS.test&set()`. If the returned value is 0, the process knows it was the first to arrive and returns its own proposal. If the returned value is 1, it returns the proposal of the other process ($1 - i$). This satisfies validity, wait-freedom, and agreement.

```
TS a test&set object init at 0
PROP array of proposals, init at whatever
```

```
propose(i, v) :=
    PROP[i] ← v
    if TS.test&set() = 0 then return PROP[i] else return PROP[1-i]
```

Listing 8.1: CN(Test&set) = 2.

Theorem 8.1.3 – CN(Test&set) $\neq 3$

However, test&set cannot solve consensus for 3 processes.

Demonstration. The proof structure is similar to the R/W proof: we find a maximally bivalent configuration C' for three processes (p, q, r). If p and q both attempt to perform a test&set on the same object, the global state results in the object being set to 1. The only difference between $p(q(C'))$ and $q(p(C'))$ is the local register of p and q (who saw the 0 and who saw the 1). If we stop p and q and let r run, r cannot distinguish between these two states and must decide the same value in both, leading to a contradiction of the valences of $p(C')$ and $q(C')$.

Other common synchronization primitives also fall into specific tiers of the hierarchy:

- *Swap and Fetch&Add (CN=2):* Like test&set, these can solve 2-process consensus by having the first process to "win" the operation return its own value and the second return the other's. For swap, a process swaps its ID into a register; if it receives the initial $\perp$, it wins. For fetch&add, the process that receives the initial 0 wins.

```
S a swap object init at ⊥
PROP array of proposals, init at whatever
```

```
propose(i, v) :=
    PROP[i] ← v
    if S.swap(i) = ⊥ then return PROP[i] else return PROP[1-i]
```

```
FA a fetch&add object init at 0
PROP array of proposals, init at whatever
```

```
propose(i, v) :=
    PROP[i] ← v
    if FA.fetch&add(1) = 0 then return PROP[i] else return PROP[1-i]
```

Listing 8.2: CN(Swap) = CN(Fetch&add) = 2.

- *Compare&Swap (CN=∞):* The compare&swap(old, new) operation is universal. It checks if the current value matches old; if so, it updates it to new and returns the actual previous value.

```
X.compare&swap(old,new) :=
    tmp ← X
    if tmp = old then X ← new
    return tmp
```

```
CS a compare&swap object init at ⊥
```

```
propose(v) :=
    tmp ← CS.compare&swap(⊥,v)
    if tmp = ⊥ then return v else return tmp
```

Listing 8.3: CN(Swap) = CN(Fetch&add) = 2.

To implement consensus for any number of processes using compare&swap, we initialize a CS object to $\perp$. Every process calls `CS.compare&swap(⊥, v)`. The very first process to execute this

will successfully change $\perp$ to its proposal v and receive $\perp$ back, thus deciding v. Every subsequent process will find that the value is no longer $\perp$, the swap will fail, and they will receive the value v proposed by the first process. Since they all receive and return the same v, consensus is achieved for any n.

Index